

ĐẠI HỌC BÁCH KHOA HÀ NỘI
KHOA TOÁN - TIN



BÁO CÁO BÀI TẬP LỚN
HỌC PHẦN: KỸ THUẬT LẬP TRÌNH

ĐỀ TÀI:
QUẢN LÝ SINH VIÊN

Giảng viên hướng dẫn:

T.S. Vũ Thành Nam

Mã lớp: 169312

Nhóm: 16

Sinh viên thực hiện:

Nguyễn Khánh Toàn – 202418996

Nguyễn Vũ Quang Anh – 202418836

Ngô Ngọc Thái – 202418984

Hà Nội, Tháng 06/2026

Mục lục

LỜI MỞ ĐẦU	1
1 GIỚI THIỆU ĐỀ TÀI	2
1.1 Giới thiệu chung	2
1.2 Mục tiêu đề tài	2
1.3 Phân chia công việc	3
2 PHÂN TÍCH YÊU CẦU	4
2.1 Yêu cầu chung	4
2.2 Yêu cầu chức năng	5
2.3 Yêu cầu phi chức năng	5
2.4 Ràng buộc kỹ thuật và dữ liệu	7
3 THIẾT KẾ CHƯƠNG TRÌNH	9
3.1 Cấu trúc thư mục dự án	9
3.2 Các module chính	12
3.3 Luồng hoạt động của chương trình	15
3.4 Thiết kế menu console	17
4 THIẾT KẾ DỮ LIỆU	20
4.1 Các struct chính	20
4.2 Quan hệ giữa các dữ liệu	23
4.3 Các file lưu trữ dữ liệu	24
4.4 Cơ chế đọc/ghi file	25
5 CẤU TRÚC DỮ LIỆU VÀ THUẬT TOÁN	27
5.1 Mảng động tự cài đặt	27
5.2 Tìm kiếm tuyến tính	29
5.3 Sắp xếp sinh viên	31
5.4 Công thức tính điểm và GPA	32

6	CÀI ĐẶT VÀ KIỂM THỬ	35
6.1	Môi trường cài đặt	35
6.2	Các chức năng đã cài đặt	37
6.3	Dữ liệu kiểm thử	39
6.4	Bảng test case	42
7	KẾT QUẢ THỰC HIỆN	46
7.1	Kiến trúc chương trình	46
7.2	Module quản lý sinh viên	46
7.3	Module quản lý môn học	47
7.4	Module quản lý lớp học phần	47
7.5	Module quản lý điểm số	48
7.6	Module đọc/ghi file	48
7.7	Module giao diện console	48
8	KẾT LUẬN	50
8.1	Kết quả đạt được	50
8.2	Hạn chế	52
8.3	Hướng phát triển	53
	TÀI LIỆU THAM KHẢO	55
	PHỤ LỤC	56
	Phụ lục A. Code hàm main	57
	Phụ lục B. Một số hàm xử lý chính	59
	Phụ lục C. File dữ liệu mẫu	69
	Phụ lục D. Một số kết quả kiểm thử	71
	Phụ lục E. Github Repository	76

LỜI MỞ ĐẦU

Trong quá trình học tập môn Kỹ thuật lập trình, việc vận dụng kiến thức lý thuyết vào xây dựng một chương trình hoàn chỉnh có ý nghĩa quan trọng đối với sinh viên. Thông qua bài tập lớn, sinh viên không chỉ rèn luyện kỹ năng lập trình bằng ngôn ngữ C mà còn có cơ hội củng cố kiến thức về tổ chức chương trình, xử lý dữ liệu, quản lý bộ nhớ, xây dựng cấu trúc dữ liệu và cài đặt các thuật toán cơ bản.

Xuất phát từ yêu cầu đó, nhóm chúng em lựa chọn đề tài *Quản lý sinh viên* dựa theo các kiến thức có sẵn và các kiến thức từ [1, 2, 3, 4, 5, 6, 7] đã được hướng dẫn ở trên lớp. Đây là một chương trình quản lý chạy trên giao diện console, cho phép thực hiện các chức năng như quản lý thông tin sinh viên, quản lý môn học, quản lý lớp học phần, nhập và cập nhật điểm số, tính điểm tổng kết, quy đổi điểm, tìm kiếm, sắp xếp và hiển thị các báo cáo cần thiết. Dữ liệu của chương trình được lưu trữ bằng các file text, qua đó giúp nhóm hiểu rõ hơn về quá trình đọc, ghi, kiểm tra và xử lý dữ liệu trong thực tế.

Do kiến thức và kinh nghiệm lập trình của nhóm còn hạn chế, bài làm khó tránh khỏi những thiếu sót trong quá trình thiết kế và triển khai. Nhóm chúng em rất mong nhận được sự góp ý của thầy để có thể hoàn thiện chương trình tốt hơn, đồng thời rút ra thêm kinh nghiệm cho các môn học và dự án sau này.

Nhóm chúng em xin chân thành cảm ơn thầy TS. Vũ Thành Nam đã hướng dẫn, định hướng và hỗ trợ nhóm trong quá trình thực hiện bài tập lớn này.

Chương 1

GIỚI THIỆU ĐỀ TÀI

1.1 Giới thiệu chung

Trong thực tế, việc quản lý thông tin sinh viên và điểm số là một công việc quan trọng trong các cơ sở đào tạo. Các thông tin như mã số sinh viên, họ tên, lớp, ngày sinh, môn học, lớp học phần và kết quả học tập cần được lưu trữ, cập nhật và tra cứu một cách chính xác. Nếu thực hiện thủ công, quá trình này dễ phát sinh sai sót, mất thời gian và khó tổng hợp khi số lượng dữ liệu tăng lên.

Xuất phát từ nhu cầu đó, nhóm chúng em thực hiện đề tài *Quản lý sinh viên*. Đây là một chương trình được xây dựng bằng ngôn ngữ lập trình C, chạy trên giao diện console, cho phép người dùng thực hiện các chức năng cơ bản như quản lý sinh viên, quản lý môn học, quản lý lớp học phần, nhập và cập nhật điểm số, tìm kiếm, sắp xếp và hiển thị bảng điểm.

Dữ liệu của chương trình được lưu trữ bằng các file text, giúp chương trình có thể đọc dữ liệu khi khởi động và ghi lại dữ liệu khi người dùng thoát. Bên cạnh đó, chương trình sử dụng các cấu trúc dữ liệu và thuật toán do nhóm tự cài đặt, phù hợp với yêu cầu của môn học Kỹ thuật lập trình. Thông qua đề tài này, nhóm có cơ hội vận dụng kiến thức về lập trình C, xử lý file, mảng động, tìm kiếm, sắp xếp và kiểm thử chương trình vào một bài toán quản lý cụ thể.

1.2 Mục tiêu đề tài

Đề tài *Quản lý sinh viên* tập trung xây dựng một chương trình hỗ trợ quản lý các thông tin cơ bản trong quá trình học tập của sinh viên. Chương trình cho phép lưu trữ và xử lý dữ liệu về sinh viên, môn học, lớp học phần và điểm số. Các thao tác chính bao

gồm thêm, sửa, xóa, tìm kiếm thông tin sinh viên; quản lý danh sách môn học, lớp học phần; nhập và cập nhật điểm cho sinh viên theo từng lớp học phần.

Bên cạnh các chức năng quản lý dữ liệu, chương trình còn hỗ trợ tính toán kết quả học tập như điểm tổng kết, điểm quy đổi hệ 4 và điểm trung bình học tập. Ngoài ra, hệ thống có thể hiển thị bảng điểm của từng sinh viên, bảng điểm của lớp học phần, đồng thời hỗ trợ sắp xếp danh sách sinh viên theo các tiêu chí như mã số sinh viên hoặc họ tên.

Chương trình được xây dựng bằng ngôn ngữ lập trình C và chạy trên giao diện console. Dữ liệu được lưu trữ trong các file text, giúp người dùng có thể lưu lại thông tin sau mỗi lần sử dụng. Trong quá trình thực hiện, nhóm tự cài đặt các cấu trúc dữ liệu và thuật toán cơ bản như mảng động, tìm kiếm tuyến tính và sắp xếp, nhằm đáp ứng yêu cầu của bài tập lớn môn Kỹ thuật lập trình.

1.3 Phân chia công việc

Trong quá trình thực hiện đề tài, nhóm phân chia công việc theo từng vai trò cụ thể nhằm đảm bảo các phần của chương trình được triển khai rõ ràng và thuận tiện cho việc tích hợp. Bảng dưới đây trình bày phân công công việc chính của từng thành viên.

Bảng 1.1: Phân công công việc nhóm

STT	Họ và tên	MSSV	Công việc chính
1	Nguyễn Khánh Toàn	202418996	Core Data, Dynamic Array, File I/O
2	Ngô Ngọc Thái	202418984	Business Logic, thuật toán, xử lý điểm
3	Nguyễn Vũ Quang Anh	202418836	Console UI, validation, kiểm thử, báo cáo

Nhìn chung, cách phân công trên giúp nhóm tách rõ các phần chính của chương trình gồm xử lý dữ liệu, xử lý nghiệp vụ và giao diện người dùng. Qua đó, các thành viên có thể triển khai song song, đồng thời dễ dàng phối hợp trong quá trình kiểm thử và hoàn thiện sản phẩm.

Chương 2

PHÂN TÍCH YÊU CẦU

2.1 Yêu cầu chung

Bài tập lớn yêu cầu nhóm xây dựng một chương trình quản lý hoàn chỉnh, có khả năng thực hiện các thao tác cơ bản thông qua menu điều khiển. Người dùng có thể lựa chọn các chức năng từ menu để thao tác với dữ liệu và chương trình chỉ kết thúc khi người dùng chọn thoát.

Chương trình được xây dựng bằng ngôn ngữ lập trình C, chạy trên giao diện console. Dữ liệu đầu vào và đầu ra của chương trình phải được lưu trữ bằng file text. Người dùng có thể nhập dữ liệu từ bàn phím, sau đó dữ liệu được xử lý trong chương trình và lưu lại xuống các file tương ứng.

Một yêu cầu quan trọng của bài tập lớn là không sử dụng các cấu trúc dữ liệu nâng cao hoặc thư viện thuật toán có sẵn. Các cấu trúc dữ liệu và thuật toán sử dụng trong chương trình cần được nhóm tự cài đặt, chẳng hạn như mảng động, thuật toán tìm kiếm và thuật toán sắp xếp. Điều này giúp nhóm vận dụng kiến thức đã học về lập trình, thiết kế chương trình, xử lý file, kiểm thử và tổ chức mã nguồn.

Đối với đề tài “Quản lý sinh viên”, chương trình cần đáp ứng các chức năng cơ bản như quản lý sinh viên, quản lý môn học, quản lý lớp học phần, quản lý điểm số, tìm kiếm, sắp xếp và hiển thị báo cáo bảng điểm. Ngoài ra, chương trình cần đảm bảo dữ liệu được lưu trữ đúng định dạng, có kiểm tra tính hợp lệ của dữ liệu nhập vào và không bị lỗi khi người dùng nhập sai.

2.2 Yêu cầu chức năng

Chương trình *Quản lý sinh viên* cần cung cấp các chức năng cơ bản phục vụ việc quản lý thông tin sinh viên, môn học, lớp học phần và kết quả học tập. Các chức năng được tổ chức thông qua giao diện menu console, giúp người dùng dễ dàng lựa chọn và thao tác.

Bảng 2.1: Các yêu cầu chức năng chính của chương trình

STT	Nhóm chức năng	Mô tả yêu cầu
1	Quản lý sinh viên	Thêm, sửa, xóa, tìm kiếm và hiển thị danh sách sinh viên. Thông tin sinh viên gồm MSSV, họ tên, lớp và ngày sinh.
2	Quản lý môn học	Thêm, sửa, xóa, tìm kiếm và hiển thị danh sách môn học. Mỗi môn học gồm mã học phần, tên học phần và số tín chỉ.
3	Quản lý lớp học phần	Thêm, sửa, xóa, tìm kiếm và hiển thị danh sách lớp học phần. Mỗi lớp học phần gắn với một mã học phần tương ứng.
4	Quản lý điểm số	Nhập và cập nhật điểm quá trình, điểm cuối kỳ cho sinh viên theo từng lớp học phần. Chương trình tự động tính điểm tổng kết và điểm hệ 4.
5	Tính kết quả học tập	Tính điểm trung bình học tập theo hệ 10, hệ 4 và hỗ trợ xếp loại học lực của sinh viên.
6	Tìm kiếm và sắp xếp	Tìm kiếm sinh viên theo MSSV, họ tên hoặc lớp; tìm kiếm môn học, lớp học phần và điểm số theo các khóa tương ứng; sắp xếp danh sách sinh viên theo MSSV, họ tên hoặc GPA.
7	Báo cáo, bảng điểm	Hiển thị bảng điểm của một sinh viên và bảng điểm của một lớp học phần để phục vụ việc theo dõi kết quả học tập.

Ngoài các chức năng trên, chương trình cần kiểm tra dữ liệu đầu vào nhằm hạn chế lỗi khi người dùng nhập sai định dạng. Các thao tác thêm, sửa, xóa dữ liệu cũng cần đảm bảo không làm mất tính nhất quán giữa sinh viên, môn học, lớp học phần và điểm số.

2.3 Yêu cầu phi chức năng

Bên cạnh các yêu cầu chức năng, chương trình *Quản lý sinh viên* cần đáp ứng một số yêu cầu phi chức năng nhằm đảm bảo chương trình hoạt động ổn định, dễ sử dụng và thuận tiện cho việc mở rộng, kiểm thử. Các yêu cầu này không trực tiếp mô tả một chức năng cụ thể, nhưng có vai trò quan trọng trong việc đánh giá chất lượng của chương trình.

Bảng 2.2: Các yêu cầu phi chức năng của chương trình

STT	Yêu cầu	Mô tả
1	Tính dễ sử dụng	Chương trình cần có menu console rõ ràng, các lựa chọn được đánh số cụ thể, thông báo dễ hiểu để người dùng thao tác thuận tiện.
2	Tính ổn định	Chương trình cần hạn chế lỗi khi người dùng nhập sai dữ liệu, không bị dừng đột ngột trong quá trình sử dụng.
3	Tính chính xác	Các thao tác thêm, sửa, xóa, tìm kiếm, sắp xếp và tính toán điểm cần cho ra kết quả đúng theo dữ liệu đầu vào và công thức đã quy định.
4	Tính nhất quán	Dữ liệu giữa sinh viên, môn học, lớp học phần và điểm số cần được kiểm tra liên kết hợp lý, tránh tình trạng dữ liệu bị trùng hoặc tham chiếu sai.
5	Tính dễ bảo trì	Mã nguồn cần được tổ chức theo module, đặt tên hàm và biến rõ ràng, có chú thích ở những phần xử lý quan trọng để thuận tiện cho việc sửa lỗi và phát triển tiếp.
6	Tính lưu trữ	Dữ liệu của chương trình cần được lưu trữ trong các file text đúng định dạng, có thể đọc lại khi chương trình khởi động và ghi lại khi người dùng thoát.
7	Tính kiểm thử	Chương trình cần có các test case để kiểm tra những chức năng chính như thêm dữ liệu, nhập điểm, tìm kiếm, sắp xếp, đọc/ghi file và xử lý dữ liệu sai.

Nhìn chung, các yêu cầu phi chức năng trên giúp chương trình không chỉ đáp ứng đúng chức năng quản lý sinh viên mà còn đảm bảo tính ổn định, rõ ràng và dễ kiểm tra. Đây cũng là những yếu tố phù hợp với định hướng của môn Kỹ thuật lập trình, trong đó nhấn mạnh việc xây dựng chương trình có cấu trúc tốt, dễ hiểu, có khả năng phòng ngừa lỗi và thuận tiện cho quá trình kiểm thử.

2.4 Ràng buộc kỹ thuật và dữ liệu

Trong quá trình xây dựng chương trình *Quản lý sinh viên*, nhóm cần tuân thủ một số ràng buộc về kỹ thuật lập trình và tổ chức dữ liệu. Các ràng buộc này nhằm đảm bảo chương trình phù hợp với yêu cầu của bài tập lớn, đồng thời giúp dữ liệu được xử lý nhất quán và an toàn hơn.

Bảng 2.3: Các ràng buộc kỹ thuật của chương trình

STT	Ràng buộc	Mô tả
1	Ngôn ngữ lập trình	Chương trình được xây dựng bằng ngôn ngữ C, chạy trên giao diện console.
2	Giao diện chương trình	Người dùng thao tác thông qua menu lập, chương trình chỉ kết thúc khi người dùng chọn chức năng thoát.
3	Lưu trữ dữ liệu	Dữ liệu được lưu trữ bằng file text, không sử dụng cơ sở dữ liệu như MySQL, SQL Server hoặc SQLite.
4	Cấu trúc dữ liệu	Nhóm tự cài đặt cấu trúc dữ liệu mảng động để lưu danh sách sinh viên, môn học, lớp học phần và điểm số.
5	Thuật toán	Các thuật toán tìm kiếm và sắp xếp được tự cài đặt, không sử dụng thư viện thuật toán có sẵn.
6	Thư viện sử dụng	Chỉ sử dụng các thư viện chuẩn của C như <code>stdio.h</code> , <code>stdlib.h</code> , <code>string.h</code> , <code>ctype.h</code> .

Bên cạnh các ràng buộc kỹ thuật, chương trình cũng cần đảm bảo các ràng buộc về dữ liệu để tránh sai lệch trong quá trình quản lý.

Bảng 2.4: Các ràng buộc dữ liệu của chương trình

STT	Ràng buộc	Mô tả
1	Khóa chính	Mã sinh viên, mã học phần và mã lớp học phần không được trùng lặp trong danh sách tương ứng.
2	Điểm số	Điểm quá trình và điểm cuối kỳ phải nằm trong khoảng từ 0 đến 10.
3	Định dạng ngày sinh	Ngày sinh của sinh viên được nhập theo định dạng DD/MM/YYYY.
4	Quan hệ dữ liệu	Mỗi lớp học phần phải gắn với một mã học phần đã tồn tại trong danh sách môn học.
5	Bản ghi điểm	Mỗi bản ghi điểm phải gắn với một sinh viên và một lớp học phần đã tồn tại. Cặp mã sinh viên và mã lớp học phần không được trùng lặp.
6	Ràng buộc khi xóa	Không cho phép xóa sinh viên nếu sinh viên đó đã có điểm; không cho phép xóa lớp học phần nếu đã có bản ghi điểm; không cho phép xóa môn học nếu đang có lớp học phần sử dụng.
7	Ký tự phân cách	Dữ liệu trong file text sử dụng ký tự để phân tách các trường, vì vậy nội dung nhập vào không được chứa ký tự này.

Những ràng buộc trên giúp chương trình hoạt động đúng theo phạm vi đề tài, đồng thời đảm bảo dữ liệu giữa các thành phần như sinh viên, môn học, lớp học phần và điểm số được quản lý một cách nhất quán.

Chương 3

THIẾT KẾ CHƯƠNG TRÌNH

3.1 Cấu trúc thư mục dự án

Dự án *Quản lý sinh viên* được tổ chức thành nhiều thư mục và file riêng biệt nhằm giúp quá trình quản lý mã nguồn, dữ liệu, tài liệu và báo cáo được rõ ràng hơn. Thay vì đặt toàn bộ chương trình trong một file duy nhất, nhóm chia dự án thành các nhóm file theo từng nhiệm vụ cụ thể như xử lý dữ liệu, đọc/ghi file, tính điểm, tìm kiếm, sắp xếp, giao diện console và kiểm thử.

Cách tổ chức này giúp chương trình có cấu trúc dễ theo dõi, đồng thời hỗ trợ quá trình phát triển theo nhóm. Mỗi thành viên có thể phụ trách một phần riêng mà không ảnh hưởng quá nhiều đến các phần còn lại. Bên cạnh đó, việc tách riêng thư mục mã nguồn, dữ liệu, ảnh minh chứng và báo cáo cũng giúp quá trình kiểm thử, sửa lỗi và hoàn thiện sản phẩm cuối cùng được thuận tiện hơn.

Trong dự án, thư mục **source/** chứa toàn bộ mã nguồn chương trình, bao gồm các file **.c**, **.h** và **Makefile**. Thư mục **data/** chứa các file dữ liệu dạng text dùng để lưu thông tin sinh viên, môn học, lớp học phân và điểm số. Ngoài ra, các thư mục như **screenshots/**, **report/** và **docs/** được sử dụng để lưu ảnh minh chứng, báo cáo và tài liệu phụ trợ.

Sơ đồ dưới đây trình bày tổng quan cấu trúc thư mục của dự án, qua đó thể hiện cách nhóm tổ chức các thành phần chính trong quá trình xây dựng chương trình.

```

QLSV_MI3310-main/
|-- source/
|   |-- main.c
|   |-- types.h
|   |-- arrays.h
|   |-- arrays.c
|   |-- student.h
|   |-- student.c
|   |-- subject.h
|   |-- subject.c
|   |-- courseclass.h
|   |-- courseclass.c
|   |-- score.h
|   |-- score.c
|   |-- gpa.h
|   |-- gpa.c
|   |-- search.h
|   |-- search.c
|   |-- sort.h
|   |-- sort.c
|   |-- fileio.h
|   |-- fileio.c
|   |-- ui.h
|   |-- ui.c
|   |-- Makefile
|   |-- test_arrays.c
|   |-- test_fileio.c
|   |-- test_fileio_unit.c
|   |-- test_gpa.c
|   |-- test_types.c
|
|-- data/
|   |-- students.txt
|   |-- subjects.txt
|   |-- course_classes.txt
|   |-- scores.txt
|
|-- screenshots/
|-- report/
|-- docs/
|-- README.md

```

Mô tả nhanh:

- **source/**: chứa mã nguồn và các file kiểm thử.
- **data/**: chứa dữ liệu lưu trữ dạng file text.
- **screenshots/**: chứa ảnh minh chứng kết quả chạy chương trình.
- **report/**: chứa báo cáo cuối kỳ.
- **docs/**: chứa tài liệu phụ trợ.
- **README.md**: mô tả dự án và hướng dẫn sử dụng.

Trong đó, thư mục `source/` là phần quan trọng nhất của dự án vì chứa toàn bộ mã nguồn chương trình. Các file trong thư mục này được chia theo từng nhóm chức năng như định nghĩa kiểu dữ liệu, cài đặt mảng động, xử lý sinh viên, môn học, lớp học phần, điểm số, tính GPA, tìm kiếm, sắp xếp, đọc/ghi file và giao diện console. Ngoài ra, thư mục này còn có một số file kiểm thử dùng để kiểm tra riêng các module quan trọng.

Thư mục `data/` chứa các file dữ liệu chính của chương trình. Dữ liệu được lưu dưới dạng file text, mỗi dòng là một bản ghi và các trường được phân tách bằng ký tự `|`. Cách lưu trữ này giúp chương trình có thể đọc dữ liệu khi khởi động và ghi lại dữ liệu khi người dùng thoát chương trình.

Bảng 3.1: Mô tả cấu trúc thư mục dự án

Thư mục/File	Mô tả
<code>source/</code>	Chứa toàn bộ mã nguồn của chương trình, bao gồm các file cài đặt <code>.c</code> , file khai báo <code>.h</code> , file kiểm thử <code>test_*.c</code> và <code>Makefile</code> để biên dịch chương trình.
<code>data/</code>	Chứa các file text dùng để lưu trữ dữ liệu sinh viên, môn học, lớp học phần và điểm số.
<code>screenshots/</code>	Chứa ảnh chụp màn hình kết quả chạy chương trình, dùng làm minh chứng trong phần kiểm thử.
<code>report/</code>	Chứa file báo cáo cuối kỳ của nhóm.
<code>docs/</code>	Chứa các tài liệu phụ trợ như ghi chú kiểm thử, kế hoạch làm việc hoặc mô tả chức năng.
<code>README.md</code>	Mô tả tổng quan dự án, mục tiêu, chức năng, phân công công việc và hướng dẫn build/chạy chương trình.

Mô tả các nhóm file trong thư mục `source/`

Các file mã nguồn trong thư mục `source/` được tổ chức theo từng nhóm chức năng cụ thể:

- **Nhóm file khởi động chương trình:** gồm `main.c`. File này là điểm bắt đầu của chương trình, thực hiện khởi tạo dữ liệu, gọi hàm đọc file, hiển thị menu chính, lưu dữ liệu và giải phóng bộ nhớ khi kết thúc.
- **Nhóm file định nghĩa dữ liệu:** gồm `types.h`. File này định nghĩa các struct chính như `Student`, `Subject`, `CourseClass`, `ScoreRecord` và các kiểu mảng động tương ứng.
- **Nhóm file cấu trúc dữ liệu:** gồm `arrays.h` và `arrays.c`. Nhóm file này cài đặt mảng động cho sinh viên, môn học, lớp học phần và điểm số.

- **Nhóm file xử lý nghiệp vụ:** gồm `student.h/.c`, `subject.h/.c`, `courseclass.h/.c`, `score.h/.c` và `gpa.h/.c`. Các file này phụ trách thao tác thêm, sửa, xóa, tìm kiếm bản ghi, xử lý điểm số và tính GPA.
- **Nhóm file thuật toán:** gồm `search.h/.c` và `sort.h/.c`. Nhóm file này cài đặt chức năng tìm kiếm sinh viên và sắp xếp danh sách sinh viên.
- **Nhóm file đọc/ghi dữ liệu:** gồm `fileio.h` và `fileio.c`. Nhóm file này phụ trách đọc dữ liệu từ các file trong thư mục `data/` và ghi dữ liệu trở lại file `text`.
- **Nhóm file giao diện:** gồm `ui.h` và `ui.c`. Nhóm file này phụ trách hiển thị menu console, nhận dữ liệu từ bàn phím, kiểm tra dữ liệu đầu vào và gọi các chức năng xử lý tương ứng.
- **Nhóm file kiểm thử:** gồm `test_arrays.c`, `test_fileio.c`, `test_fileio_unit.c`, `test_gpa.c` và `test_types.c`. Các file này được dùng để kiểm tra riêng từng module quan trọng, từ đó giúp nhóm phát hiện lỗi sớm và đảm bảo chương trình hoạt động ổn định hơn.
- **File Makefile:** hỗ trợ biên dịch chương trình nhanh chóng bằng lệnh `make`, giúp giảm thao tác nhập lệnh biên dịch thủ công.

Việc tổ chức thư mục như trên giúp dự án có cấu trúc rõ ràng, dễ quản lý và thuận tiện cho quá trình phát triển, kiểm thử cũng như hoàn thiện sản phẩm cuối cùng.

3.2 Các module chính

Chương trình được thiết kế theo hướng module hóa, trong đó mỗi file hoặc nhóm file đảm nhiệm một vai trò riêng. Cách tổ chức này giúp mã nguồn rõ ràng hơn, hạn chế việc tập trung toàn bộ xử lý vào một file lớn, đồng thời thuận tiện cho quá trình kiểm thử, sửa lỗi và mở rộng chương trình.

```
main.c
|-- types.h           : Khai bao cac kieu du lieu chinh
|-- arrays.h/.c       : Cai dat mang dong cho cac kieu du lieu
|-- student.h/.c      : Xu ly nghiep vu sinh vien
|-- subject.h/.c      : Xu ly nghiep vu mon hoc
|-- courseclass.h/.c  : Xu ly nghiep vu lop hoc phan
|-- score.h/.c        : Xu ly diem so va quy doi diem he 4
|-- gpa.h/.c          : Tinh GPA cua sinh vien
|-- search.h/.c       : Tim kiem sinh vien
|-- sort.h/.c         : Sap xep danh sach sinh vien
|-- fileio.h/.c       : Doc va ghi du lieu file text
```

```
|-- ui.h/.c           : Giao diện console và xử lý nhập liệu
|-- test_*.c         : Các file kiểm thử module
```

Cụ thể, các module chính trong chương trình gồm:

- **Module main.c:** là điểm bắt đầu của chương trình. File này thực hiện khởi tạo các mảng dữ liệu, gọi hàm đọc dữ liệu từ file, hiển thị menu chính, lưu dữ liệu khi người dùng thoát và giải phóng bộ nhớ đã cấp phát.
- **Module types.h:** phụ trách định nghĩa các kiểu dữ liệu chính của chương trình. File này khai báo các struct biểu diễn sinh viên, môn học, lớp học phần và điểm số, gồm `Student`, `Subject`, `CourseClass` và `ScoreRecord`. Ngoài ra, `types.h` còn định nghĩa các kiểu mảng động tương ứng như `StudentArray`, `SubjectArray`, `CourseClassArray` và `ScoreArray` để sử dụng thống nhất trong toàn bộ chương trình.
- **Module arrays.h/.c:** cài đặt cấu trúc dữ liệu mảng động cho từng nhóm dữ liệu. Mỗi mảng động gồm con trỏ `data`, biến `size` và biến `capacity`. Module này cung cấp các thao tác cơ bản như khởi tạo, thêm phần tử, lấy phần tử, cập nhật, xóa, tìm kiếm và giải phóng bộ nhớ. Khi mảng đầy, chương trình tự động mở rộng vùng nhớ bằng cách tăng `capacity`.
- **Module student.h/.c:** phụ trách xử lý nghiệp vụ liên quan đến sinh viên. Module này cung cấp các hàm thêm, cập nhật, xóa và tìm kiếm sinh viên. Các thao tác được thực hiện thông qua `StudentArray`, trong đó mã số sinh viên `MSSV` được sử dụng làm khóa chính để xác định từng sinh viên.
- **Module subject.h/.c:** phụ trách xử lý nghiệp vụ liên quan đến môn học. Module này cung cấp các hàm thêm, cập nhật, xóa và tìm kiếm môn học. Mỗi môn học được xác định thông qua mã học phần `MaHP`. Khi thêm môn học mới, chương trình kiểm tra mã học phần để tránh trùng lặp dữ liệu.
- **Module courseclass.h/.c:** phụ trách xử lý nghiệp vụ liên quan đến lớp học phần. Module này cung cấp các hàm thêm, cập nhật, xóa và tìm kiếm lớp học phần. Khi thêm lớp học phần mới, chương trình kiểm tra mã lớp học phần `MaLHP` để tránh trùng lặp, đồng thời kiểm tra mã học phần `MaHP` có tồn tại trong danh sách môn học hay không.
- **Module score.h/.c:** phụ trách xử lý điểm số của sinh viên theo từng lớp học phần. Module này cài đặt các hàm tính điểm tổng kết, quy đổi điểm hệ 4, thêm bản ghi điểm và cập nhật điểm. Khi thêm điểm, chương trình kiểm tra sinh viên và lớp học phần có tồn tại hay không, đồng thời kiểm tra cặp khóa (`MSSV`, `MaLHP`) để tránh nhập trùng điểm cho cùng một sinh viên trong cùng một lớp học phần.

- **Module gpa.h/.c:** phụ trách tính GPA của sinh viên. Module này sử dụng dữ liệu từ `ScoreArray`, `CourseClassArray` và `SubjectArray` để liên kết điểm số với lớp học phần và môn học. Từ đó, chương trình lấy số tín chỉ tương ứng và tính GPA theo trọng số tín chỉ dựa trên điểm hệ 4.
- **Module search.h/.c:** phụ trách các chức năng tìm kiếm tuyến tính trên các danh sách dữ liệu. Module này hỗ trợ tìm sinh viên theo MSSV, họ tên hoặc lớp; tìm môn học theo mã học phần hoặc tên học phần; và tìm lớp học phần theo mã lớp học phần. Các hàm tìm kiếm đều duyệt tuần tự trên mảng động tương ứng, phù hợp với quy mô dữ liệu của bài tập lớn.
- **Module sort.h/.c:** chứa các hàm sắp xếp danh sách sinh viên. Chương trình hỗ trợ sắp xếp sinh viên theo mã số sinh viên và theo họ tên. Các hàm sắp xếp được tự cài đặt bằng thuật toán Bubble Sort, sử dụng `strcmp` để so sánh chuỗi và hoán đổi vị trí hai sinh viên khi cần thiết.
- **Module fileio.h/.c:** phụ trách việc đọc và ghi dữ liệu từ các file text. Module này cài đặt các hàm đọc/ghi riêng cho sinh viên, môn học, lớp học phần và điểm số. Ngoài ra, hai hàm `loadAllData` và `saveAllData` được sử dụng để đọc hoặc ghi toàn bộ dữ liệu của chương trình trong một lần. Trong quá trình đọc file, chương trình có kiểm tra các lỗi như thiếu trường, trùng khóa, điểm ngoài khoảng hợp lệ và khóa ngoại không tồn tại.
- **Module ui.h/.c:** phụ trách giao diện console và xử lý tương tác với người dùng. Module này hiển thị menu, nhận dữ liệu từ bàn phím, kiểm tra dữ liệu đầu vào, gọi các chức năng quản lý và hiển thị kết quả. Đây là module kết nối giữa người dùng và các module xử lý dữ liệu bên trong chương trình.
- **Các file kiểm thử test_*.c:** được sử dụng để kiểm tra tính đúng đắn của một số module quan trọng. Ví dụ, `test_arrays.c` kiểm thử mảng động, `test_fileio.c` kiểm thử đọc/ghi dữ liệu và toàn vẹn khóa ngoại, `test_gpa.c` kiểm thử hàm tính GPA, còn `test_types.c` kiểm thử các struct dữ liệu chính.

Nhìn chung, việc chia chương trình thành các module giúp quá trình phát triển được rõ ràng hơn. Mỗi module tập trung vào một nhiệm vụ cụ thể, nhờ đó chương trình dễ đọc, dễ kiểm thử và dễ bảo trì hơn. Cách tổ chức này cũng giúp nhóm thuận tiện hơn khi phân công công việc, sửa lỗi và mở rộng chức năng trong quá trình hoàn thiện bài tập lớn.

3.2.1 Liên hệ với kỹ thuật thiết kế chương trình

Cách tổ chức chương trình của nhóm thể hiện rõ tư tưởng module hóa trong thiết kế chương trình. Thay vì viết toàn bộ chức năng trong một file duy nhất, chương trình được

chia thành nhiều module nhỏ, mỗi module phụ trách một nhóm nhiệm vụ cụ thể. Ví dụ, `arrays.c` phụ trách cấu trúc dữ liệu mảng động, `fileio.c` phụ trách đọc/ghi dữ liệu, `score.c` phụ trách xử lý điểm số, còn `ui.c` phụ trách giao diện console và nhập liệu.

Cách chia này giúp bài toán lớn được tách thành các bài toán nhỏ hơn. Đây là cách tiếp cận phù hợp với nguyên tắc thiết kế chương trình theo hướng từ trên xuống: trước hết xác định các nhiệm vụ chính của hệ thống, sau đó tinh chỉnh từng nhiệm vụ thành các hàm và module cụ thể. Nhờ đó, chương trình dễ đọc hơn, dễ phân công công việc trong nhóm hơn và thuận tiện hơn khi kiểm thử từng phần.

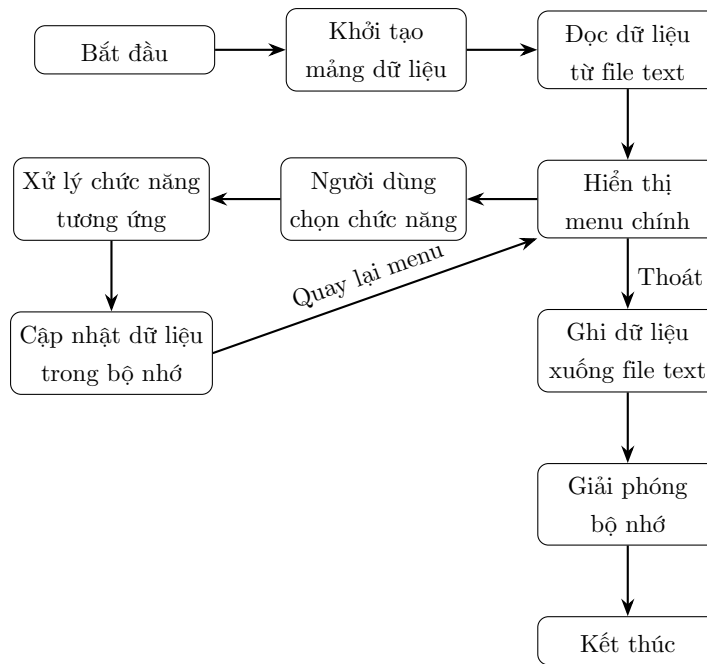
Ngoài ra, việc tách phần xử lý nghiệp vụ khỏi phần giao diện cũng giúp chương trình có tính bảo trì cao hơn. Các module như `student.c`, `subject.c`, `courseclass.c` và `score.c` chỉ xử lý dữ liệu, trong khi `ui.c` chịu trách nhiệm nhập xuất và kiểm tra đầu vào. Khi cần thay đổi giao diện hoặc bổ sung cách nhập dữ liệu mới, nhóm có thể sửa ở tầng giao diện mà ít ảnh hưởng đến các hàm xử lý nghiệp vụ bên trong.

3.3 Luồng hoạt động của chương trình

Chương trình được thiết kế theo luồng xử lý tuần tự, bắt đầu từ hàm `main`. Khi chương trình khởi động, các mảng dữ liệu chính như danh sách sinh viên, môn học, lớp học phần và điểm số được khởi tạo trong bộ nhớ. Sau đó, chương trình đọc dữ liệu từ các file text trong thư mục `data/` và nạp vào các mảng động tương ứng.

Sau khi dữ liệu đã được nạp, chương trình hiển thị menu chính để người dùng lựa chọn chức năng cần thực hiện. Từ menu chính, người dùng có thể chuyển đến các nhóm chức năng như quản lý sinh viên, quản lý môn học, quản lý lớp học phần, quản lý điểm số hoặc xem báo cáo bảng điểm. Sau mỗi thao tác, dữ liệu được cập nhật trực tiếp trong bộ nhớ và chương trình quay lại menu để tiếp tục nhận lựa chọn mới.

Các thay đổi trong quá trình chạy chương trình chưa được ghi ngay xuống file sau từng thao tác. Dữ liệu chỉ được lưu lại khi người dùng chọn chức năng lưu và thoát. Khi đó, chương trình ghi toàn bộ dữ liệu hiện tại xuống các file text, giải phóng bộ nhớ đã cấp phát và kết thúc chương trình. Luồng hoạt động tổng quát của chương trình được mô tả trong sơ đồ dưới đây.



Cụ thể, khi người dùng chọn một chức năng trong menu, chương trình sẽ chuyển đến submenu tương ứng. Tại mỗi submenu, người dùng có thể thực hiện các thao tác như thêm, sửa, xóa, tìm kiếm, sắp xếp hoặc hiển thị dữ liệu. Ví dụ, trong nhóm chức năng quản lý sinh viên, người dùng có thể thêm sinh viên mới, cập nhật thông tin sinh viên, tìm kiếm sinh viên theo mã số hoặc họ tên và sắp xếp danh sách sinh viên.

Trong quá trình xử lý, dữ liệu được thao tác trực tiếp trên các mảng động trong bộ nhớ. Điều này giúp các thao tác thêm, sửa, xóa và tìm kiếm được thực hiện nhanh chóng trong phiên làm việc hiện tại. Khi người dùng chọn thoát, chương trình mới ghi dữ liệu từ bộ nhớ xuống các file text để đảm bảo dữ liệu được lưu lại cho lần chạy sau.

Cách tổ chức luồng hoạt động như trên giúp chương trình đơn giản, dễ kiểm soát và phù hợp với quy mô của bài tập lớn. Đồng thời, việc tách riêng các bước khởi tạo dữ liệu, đọc file, xử lý menu, lưu file và giải phóng bộ nhớ cũng giúp chương trình rõ ràng hơn trong quá trình kiểm thử và bảo trì.

3.4 Thiết kế menu console

Chương trình sử dụng giao diện console với hệ thống menu phân cấp. Người dùng thao tác bằng cách nhập số tương ứng với chức năng cần thực hiện. Menu chính được lặp lại trong suốt quá trình chạy chương trình và chỉ kết thúc khi người dùng chọn chức năng lưu và thoát.

Việc thiết kế menu theo dạng phân cấp giúp chương trình đơn giản, dễ sử dụng và phù hợp với phạm vi của bài tập lớn. Các chức năng được chia thành từng nhóm rõ ràng như quản lý sinh viên, quản lý môn học, quản lý lớp học phần, quản lý điểm số và báo cáo kết quả học tập. Cách chia này giúp người dùng dễ tìm chức năng cần sử dụng, đồng thời giúp mã nguồn được tổ chức theo từng nhóm xử lý riêng.

Bên cạnh đó, thiết kế menu console cũng phù hợp với cách tổ chức module của chương trình. Khi người dùng chọn một chức năng trên menu, module giao diện sẽ tiếp nhận lựa chọn, kiểm tra dữ liệu nhập vào nếu cần, sau đó gọi đến các module xử lý tương ứng như sinh viên, môn học, lớp học phần, điểm số, tìm kiếm, sắp xếp hoặc đọc/ghi file.

```
===== STUDENT MANAGEMENT SYSTEM =====
1. Quan ly sinh vien
2. Quan ly mon hoc
3. Quan ly lop hoc phan
4. Quan ly diem so
5. Bao cao / bang diem
6. Hien thi tat ca du lieu
0. Luu va thoat
Nhap lua chon: █
```

Hình 3.1: Giao diện menu chính của chương trình

MENU CHÍNH

```
|-- 1. Quan ly sinh vien
|   |-- Hien thi danh sach sinh vien
|   |-- Them sinh vien
|   |-- Sua thong tin sinh vien
|   |-- Xoa sinh vien
|   |-- Tim kiem sinh vien
|   |-- Sap xep danh sach sinh vien
|
|-- 2. Quan ly mon hoc
|   |-- Hien thi danh sach mon hoc
|   |-- Them mon hoc
|   |-- Sua thong tin mon hoc
|   |-- Xoa mon hoc
|   |-- Tim kiem mon hoc
|
|-- 3. Quan ly lop hoc phan
|   |-- Hien thi danh sach lop hoc phan
|   |-- Them lop hoc phan
|   |-- Sua thong tin lop hoc phan
|   |-- Xoa lop hoc phan
|   |-- Tim kiem lop hoc phan
|
|-- 4. Quan ly diem so
|   |-- Hien thi danh sach diem
|   |-- Nhap diem sinh vien
|   |-- Cap nhat diem sinh vien
|   |-- Tim kiem diem
|
|-- 5. Bao cao / bang diem
|   |-- Bang diem cua mot sinh vien
|   |-- Bang diem cua mot lop hoc phan
|   |-- Tinh GPA cua sinh vien
|
|-- 6. Hien thi tat ca du lieu
|-- 0. Luu va thoat
```

Ở menu chính, người dùng có thể chọn từng nhóm chức năng tương ứng. Khi chọn một nhóm chức năng, chương trình sẽ chuyển sang menu con của nhóm đó. Sau khi thực hiện xong thao tác, chương trình quay lại menu trước đó để người dùng tiếp tục sử dụng hoặc chọn thoát. Cách tổ chức này tạo thành một vòng lặp xử lý rõ ràng, giúp chương trình dễ kiểm soát trong quá trình chạy.

Trong quá trình nhập dữ liệu, chương trình thực hiện kiểm tra dữ liệu đầu vào để hạn chế lỗi. Ví dụ, mã sinh viên, mã học phần và mã lớp học phần không được để trống; điểm số phải nằm trong khoảng hợp lệ; ngày sinh cần đúng định dạng; học kỳ và năm học phải hợp lý. Ngoài ra, dữ liệu nhập vào không được chứa ký tự phân cách | để tránh làm sai định dạng khi ghi dữ liệu xuống file text.

Đối với các thao tác có liên quan đến nhiều nhóm dữ liệu, chương trình cũng kiểm tra sự tồn tại của dữ liệu liên kết. Ví dụ, khi thêm lớp học phần, mã học phần phải tồn tại trong danh sách môn học; khi nhập điểm, mã số sinh viên và mã lớp học phần phải hợp lệ; khi thêm bản ghi điểm, cặp (MSSV, MaLHP) không được trùng lặp. Điều này giúp chương trình hạn chế dữ liệu sai và đảm bảo tính nhất quán giữa các file lưu trữ.

Nhìn chung, thiết kế menu console của chương trình được xây dựng theo hướng đơn giản, rõ ràng và dễ thao tác. Các nhóm chức năng được phân chia hợp lý, phù hợp với yêu cầu của đề tài *Quản lý sinh viên*. Đồng thời, cách thiết kế này cũng giúp quá trình kiểm thử thuận tiện hơn vì có thể kiểm tra riêng từng nhóm chức năng trong chương trình.

Chương 4

THIẾT KẾ DỮ LIỆU

4.1 Các struct chính

Trong chương trình *Quản lý sinh viên*, dữ liệu được tổ chức thông qua các **struct** trong ngôn ngữ C. Mỗi **struct** biểu diễn một nhóm thông tin cụ thể như sinh viên, môn học, lớp học phần hoặc điểm số. Việc sử dụng **struct** giúp dữ liệu có cấu trúc rõ ràng hơn, thuận tiện cho quá trình lưu trữ, xử lý và truyền dữ liệu giữa các module trong chương trình.

Các **struct** chính của chương trình được khai báo trong file **types.h**. Đây là file nền tảng, được các module khác sử dụng để đảm bảo toàn bộ chương trình dùng thống nhất một kiểu dữ liệu. Bốn struct chính được sử dụng trong chương trình gồm **Student**, **Subject**, **CourseClass** và **ScoreRecord**.

Struct Student

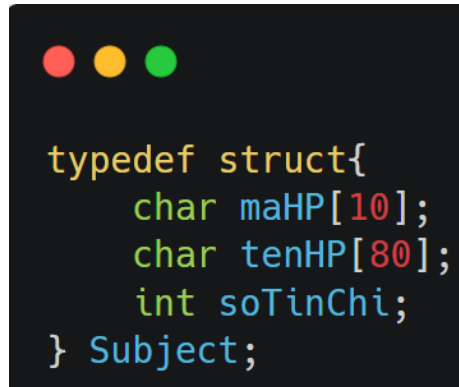
A screenshot of a code editor with a dark background and three colored window control buttons (red, yellow, green) in the top left corner. The code defines a C struct named 'Student' using 'typedef struct'. It contains four character arrays: 'mssv' with size 12, 'hoTen' with size 60, 'lop' with size 20, and 'birthday' with size 12. The struct is closed with a brace and named 'Student'.

```
typedef struct{
    char mssv[12];
    char hoTen[60];
    char lop[20];
    char birthday[12];
} Student;
```

Hình 4.1: Struct Student

Student dùng để lưu trữ thông tin cơ bản của một sinh viên. Trong đó, **mssv** là mã số sinh viên và được xem là khóa chính để phân biệt các sinh viên với nhau. Các trường còn lại gồm **hoTen**, **lop** và **birthday**, tương ứng với họ tên, lớp và ngày sinh của sinh viên.

Struct Subject

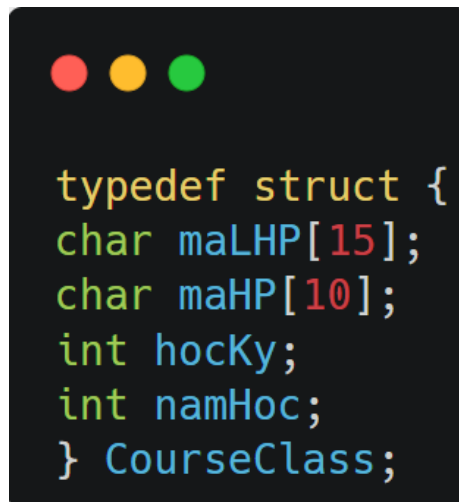


```
typedef struct{
    char maHP[10];
    char tenHP[80];
    int soTinChi;
} Subject;
```

Hình 4.2: Struct Subject

Subject dùng để lưu thông tin của một môn học. Mỗi môn học gồm mã học phần, tên học phần và số tín chỉ. Trường **maHP** được dùng làm khóa chính để quản lý và tìm kiếm môn học. Trường **soTinChi** có vai trò quan trọng trong quá trình tính GPA vì điểm trung bình được tính theo trọng số tín chỉ.

Struct CourseClass

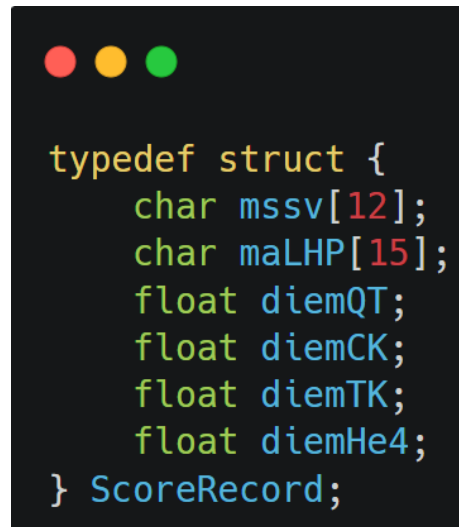


```
typedef struct {
    char maLHP[15];
    char maHP[10];
    int hocKy;
    int namHoc;
} CourseClass;
```

Hình 4.3: Struct CourseClass

CourseClass biểu diễn thông tin của một lớp học phần. Mỗi lớp học phần có mã lớp học phần riêng **maLHP**, đồng thời liên kết với một môn học thông qua trường **maHP**. Ngoài ra, struct này còn lưu thông tin về học kỳ và năm học của lớp học phần. Nhờ đó, chương trình có thể quản lý cùng một môn học được mở ở nhiều lớp học phần khác nhau.

Struct ScoreRecord



```
typedef struct {  
    char mssv[12];  
    char maLHP[15];  
    float diemQT;  
    float diemCK;  
    float diemTK;  
    float diemHe4;  
} ScoreRecord;
```

Hình 4.4: Struct ScoreRecord

ScoreRecord dùng để lưu điểm của một sinh viên trong một lớp học phần. Mỗi bản ghi điểm gồm mã sinh viên, mã lớp học phần, điểm quá trình, điểm cuối kỳ, điểm tổng kết và điểm quy đổi hệ 4. Cặp **mssv** và **maLHP** được sử dụng để xác định duy nhất một bản ghi điểm, tránh trường hợp một sinh viên có nhiều bản ghi điểm trùng nhau trong cùng một lớp học phần.

Các struct mảng động

Bên cạnh các struct biểu diễn dữ liệu nghiệp vụ, chương trình còn định nghĩa các struct mảng động như **StudentArray**, **SubjectArray**, **CourseClassArray** và **ScoreArray**. Mỗi mảng động gồm ba thành phần chính:

- **data**: con trỏ trỏ đến vùng nhớ lưu các phần tử.
- **size**: số lượng phần tử hiện có trong mảng.
- **capacity**: sức chứa hiện tại của mảng.

Các struct mảng động này là cơ sở để chương trình tự cài đặt các thao tác thêm, sửa, xóa, tìm kiếm và giải phóng bộ nhớ cho từng nhóm dữ liệu. Nhờ đó, chương trình không cần dùng mảng tĩnh có kích thước cố định mà vẫn có thể quản lý dữ liệu linh hoạt trong quá trình chạy.

Nhìn chung, các **struct** trên là nền tảng dữ liệu chính của chương trình. Chúng giúp biểu diễn rõ các đối tượng cần quản lý, đồng thời tạo cơ sở để xây dựng mảng động, chức năng đọc ghi file, tìm kiếm, sắp xếp, xử lý điểm số và hiển thị dữ liệu.

4.2 Quan hệ giữa các dữ liệu

Trong chương trình *Quản lý sinh viên*, các dữ liệu không tồn tại độc lập mà có mối quan hệ với nhau thông qua các khóa định danh. Cụ thể, sinh viên được xác định bằng mã số sinh viên, môn học được xác định bằng mã học phần, lớp học phần được xác định bằng mã lớp học phần và bản ghi điểm liên kết sinh viên với lớp học phần tương ứng.



Quan hệ giữa các dữ liệu có thể mô tả như sau:

- **Sinh viên và điểm số:** mỗi sinh viên có thể có nhiều bản ghi điểm khác nhau. Trường MSSV trong **ScoreRecord** dùng để liên kết bản ghi điểm với sinh viên tương ứng.
- **Môn học và lớp học phần:** mỗi môn học có thể được mở thành nhiều lớp học phần. Trường MaHP trong **CourseClass** dùng để xác định lớp học phần đó thuộc môn học nào.
- **Lớp học phần và điểm số:** mỗi lớp học phần có thể có nhiều sinh viên tham gia và có nhiều bản ghi điểm. Trường MaLHP trong **ScoreRecord** dùng để liên kết điểm số với lớp học phần tương ứng.
- **Bản ghi điểm:** một bản ghi điểm được xác định duy nhất bởi cặp MSSV và MaLHP. Điều này đảm bảo một sinh viên chỉ có một bản ghi điểm trong cùng một lớp học phần.

Như vậy, **ScoreRecord** đóng vai trò là dữ liệu trung gian liên kết giữa sinh viên và lớp học phần. Thông qua lớp học phần, chương trình có thể xác định được môn học tương ứng và số tín chỉ cần dùng khi tính điểm trung bình. Cách thiết kế này giúp dữ liệu được tổ chức rõ ràng, hạn chế trùng lặp và thuận tiện cho các thao tác như nhập điểm, tìm kiếm bảng điểm, tính GPA và kiểm tra ràng buộc khi xóa dữ liệu.

Để đảm bảo tính nhất quán dữ liệu, chương trình cần kiểm tra các ràng buộc trước khi thao tác. Cụ thể, không cho phép nhập điểm cho sinh viên hoặc lớp học phần không tồn tại; không cho phép xóa sinh viên đã có điểm; không cho phép xóa lớp học phần đã có bản ghi điểm; và không cho phép xóa môn học nếu vẫn còn lớp học phần đang sử dụng môn học đó.

4.3 Các file lưu trữ dữ liệu

Dữ liệu của chương trình được lưu trữ trong các file text thuộc thư mục **data/**. Thay vì sử dụng cơ sở dữ liệu, nhóm lựa chọn cách lưu dữ liệu bằng file văn bản để phù hợp với yêu cầu của bài tập lớn và thuận tiện cho việc đọc, ghi dữ liệu bằng ngôn ngữ C.

Các file dữ liệu sử dụng định dạng đơn giản, trong đó mỗi dòng biểu diễn một bản ghi. Các trường dữ liệu trong cùng một dòng được phân tách bằng ký tự `|`. Dòng đầu tiên của mỗi file là dòng tiêu đề, dùng để mô tả tên các trường dữ liệu.

data/

```
-- students.txt      : Lưu thông tin sinh viên
-- subjects.txt      : Lưu thông tin môn học
-- course_classes.txt : Lưu thông tin lớp học phần
-- scores.txt        : Lưu thông tin điểm số
```

File students.txt

File **students.txt** dùng để lưu thông tin sinh viên. Mỗi sinh viên gồm mã số sinh viên, họ tên, lớp và ngày sinh.

```
MSSV|HoTen|Lop|Birthday
202400000|Nguyen Van Toan|K69-MI1-01|15/08/2006
202400001|Tran Quan Anh|K69-MI1-02|20/03/2006
```

Trong đó, **MSSV** là khóa chính dùng để phân biệt các sinh viên. Trường này không được để trống và không được trùng lặp.

File subjects.txt

File **subjects.txt** dùng để lưu thông tin môn học. Mỗi môn học gồm mã học phần, tên học phần và số tín chỉ.

```
MaHP|TenHP|SoTinChi
MI3310|Ky Thuat Lap Trinh|2
MI3060|Cau Truc Du Lieu & Thuat Toan|3
```

Trường **MaHP** là khóa chính của môn học. Mỗi môn học có một mã học phần riêng và có thể được sử dụng để tạo các lớp học phần tương ứng.

File `course_classes.txt`

File `course_classes.txt` dùng để lưu thông tin lớp học phần. Mỗi lớp học phần gồm mã lớp học phần, mã học phần, học kỳ và năm học.

```
MaLHP|MaHP|HocKy|NamHoc  
169313|MI3310|1|2025  
169307|MI3060|2|2025
```

Trường `MaLHP` là khóa chính của lớp học phần. Trường `MaHP` dùng để liên kết lớp học phần với môn học tương ứng trong file `subjects.txt`.

File `scores.txt`

File `scores.txt` dùng để lưu điểm của sinh viên theo từng lớp học phần. Mỗi bản ghi điểm gồm mã sinh viên, mã lớp học phần, điểm quá trình, điểm cuối kỳ, điểm tổng kết và điểm hệ 4.

```
MSSV|MaLHP|DiemQT|DiemCK|DiemTK|DiemHe4  
202400000|169313|8.0|7.5|7.75|3.0  
202400001|169307|7.0|8.0|7.50|3.0
```

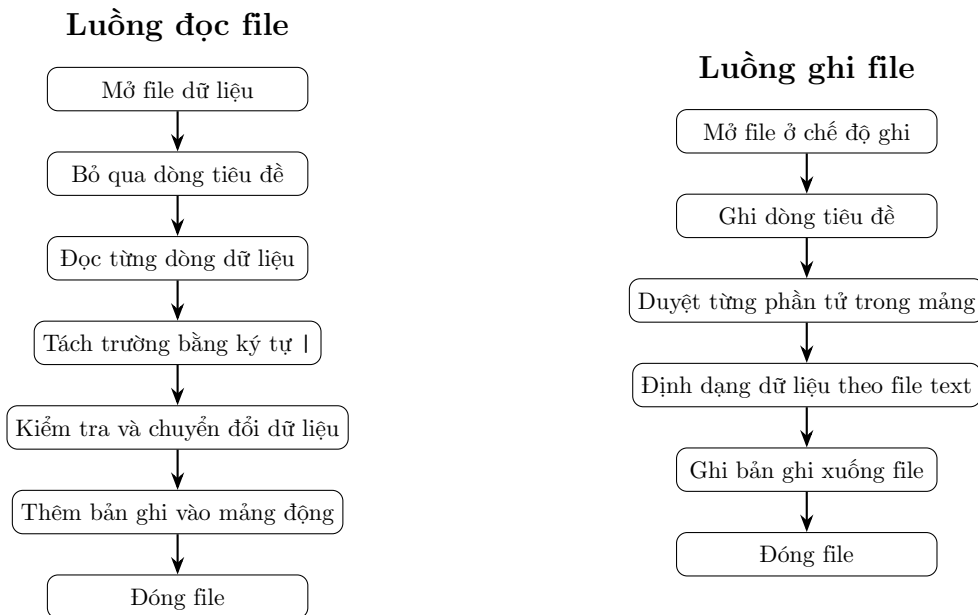
Trong file này, cặp `MSSV` và `MaLHP` được dùng để xác định duy nhất một bản ghi điểm. Điều này đảm bảo một sinh viên chỉ có một bản ghi điểm trong cùng một lớp học phần.

Nhìn chung, cách tổ chức dữ liệu bằng các file text giúp chương trình đơn giản, dễ kiểm tra và phù hợp với phạm vi của bài tập lớn. Khi chương trình khởi động, dữ liệu được đọc từ các file trên vào bộ nhớ. Khi người dùng chọn thoát, dữ liệu hiện tại sẽ được ghi lại xuống file để sử dụng cho các lần chạy tiếp theo.

4.4 Cơ chế đọc/ghi file

Trong chương trình *Quản lý sinh viên*, dữ liệu được lưu trữ bằng các file text trong thư mục `data/`. Khi chương trình khởi động, dữ liệu được đọc từ file vào các mảng động trong bộ nhớ. Trong quá trình sử dụng, người dùng thao tác trực tiếp với dữ liệu trong bộ nhớ. Khi người dùng chọn thoát chương trình, toàn bộ dữ liệu hiện tại sẽ được ghi lại xuống các file text tương ứng.

Cơ chế đọc/ghi file của chương trình được chia thành hai luồng chính: luồng đọc file và luồng ghi file.



Ở luồng đọc file, chương trình mở lần lượt các file dữ liệu như `students.txt`, `subjects.txt`, `course_classes.txt` và `scores.txt`. Sau khi bỏ qua dòng tiêu đề, chương trình đọc từng dòng, tách các trường bằng ký tự `|`, kiểm tra dữ liệu và đưa bản ghi hợp lệ vào mảng động tương ứng.

Ở luồng ghi file, khi người dùng chọn thoát chương trình, dữ liệu trong các mảng động sẽ được ghi lại xuống file. Chương trình ghi dòng tiêu đề trước, sau đó duyệt từng phần tử trong mảng và ghi dữ liệu theo đúng định dạng đã quy định.

Cách tổ chức này giúp chương trình có thể lưu trữ dữ liệu giữa các lần chạy mà không cần sử dụng cơ sở dữ liệu. Đồng thời, việc đọc dữ liệu vào bộ nhớ rồi xử lý trên mảng động cũng giúp chương trình đơn giản, dễ kiểm soát và phù hợp với phạm vi của bài tập lớn.

Chương 5

CẤU TRÚC DỮ LIỆU VÀ THUẬT TOÁN

Trong học phần Kỹ thuật lập trình, khi giải quyết một bài toán thực tế, người lập trình cần xác định rõ các dữ liệu liên quan và các thao tác cần thiết trên dữ liệu đó. Với đề tài này, dữ liệu chính gồm sinh viên, môn học, lớp học phần và điểm số. Tương ứng với các dữ liệu đó, chương trình cần có các thao tác như thêm, sửa, xóa, tìm kiếm, sắp xếp, đọc/ghi file và tính toán kết quả học tập.

Do đó, chương này trình bày các cấu trúc dữ liệu và thuật toán được nhóm sử dụng trong chương trình. Các nội dung chính gồm mảng động tự cài đặt, tìm kiếm tuyến tính, sắp xếp sinh viên và các công thức tính điểm, GPA. Đây đều là các nội dung gắn trực tiếp với yêu cầu của bài tập lớn và kiến thức lý thuyết của học phần.

5.1 Mảng động tự cài đặt

Trong chương trình *Quản lý sinh viên*, nhóm sử dụng cấu trúc dữ liệu mảng động tự cài đặt để lưu trữ các danh sách dữ liệu như sinh viên, môn học, lớp học phần và điểm số. Việc sử dụng mảng động giúp chương trình có thể thay đổi kích thước lưu trữ trong quá trình chạy, thay vì bị giới hạn bởi một kích thước cố định như mảng tĩnh.

Mỗi mảng động trong chương trình gồm ba thành phần chính:

- **data:** con trỏ trỏ đến vùng nhớ chứa các phần tử.
- **size:** số lượng phần tử hiện có trong mảng.
- **capacity:** sức chứa tối đa hiện tại của mảng.

Ví dụ, mảng động dùng để lưu danh sách sinh viên được khai báo như sau:

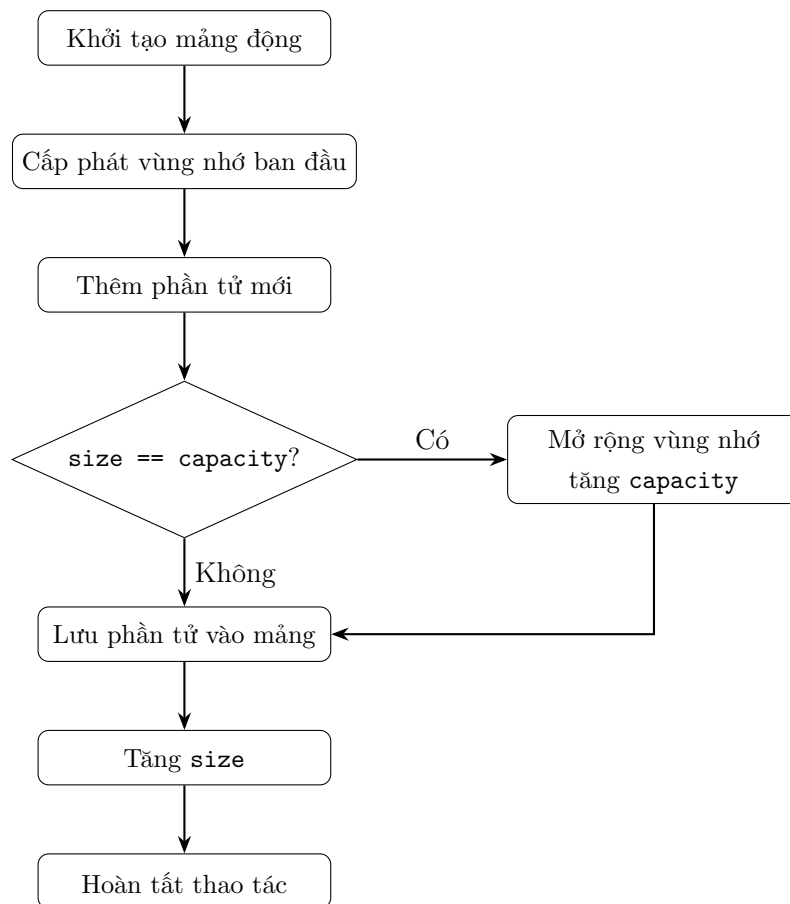
```

typedef struct {
    Student* data;
    int size;
    int capacity;
} StudentArray;

```

Tương tự, chương trình cũng xây dựng các mảng động khác như `SubjectArray`, `CourseClassArray` và `ScoreArray` để lưu trữ môn học, lớp học phần và điểm số. Cách thiết kế này giúp mỗi loại dữ liệu có một mảng riêng, dễ sử dụng và tránh nhầm lẫn kiểu dữ liệu trong quá trình xử lý.

Cơ chế hoạt động của mảng động được mô tả qua sơ đồ luồng sau:



Khi thêm một phần tử mới, chương trình kiểm tra xem `size` đã bằng `capacity` hay chưa. Nếu mảng đã đầy, chương trình sẽ cấp phát lại vùng nhớ với dung lượng lớn hơn, thường là gấp đôi dung lượng cũ. Sau đó, phần tử mới được thêm vào cuối mảng và giá trị `size` được tăng lên một đơn vị.

Các thao tác chính trên mảng động gồm:

- `init`: khởi tạo mảng động và cấp phát vùng nhớ ban đầu.

- **add**: thêm phần tử mới vào mảng.
- **get**: lấy phần tử theo chỉ số.
- **update**: cập nhật thông tin phần tử.
- **remove**: xóa phần tử khỏi mảng.
- **find**: tìm kiếm phần tử theo khóa.
- **clear**: giải phóng vùng nhớ khi không còn sử dụng.

Ví dụ, với danh sách sinh viên, chương trình sử dụng các hàm như `sa_init`, `sa_add`, `sa_get`, `sa_remove`, `sa_update`, `sa_find` và `sa_clear`. Các danh sách khác cũng được xây dựng tương tự với các tiền tố khác nhau như `suba_` cho môn học, `cca_` cho lớp học phần và `sca_` cho điểm số.

Việc tự cài đặt mảng động giúp nhóm hiểu rõ hơn về cách quản lý bộ nhớ trong ngôn ngữ C, đặc biệt là quá trình cấp phát, mở rộng và giải phóng vùng nhớ. Đồng thời, cấu trúc dữ liệu này cũng đáp ứng yêu cầu của bài tập lớn là không sử dụng các thư viện cấu trúc dữ liệu có sẵn, mà phải tự xây dựng cấu trúc dữ liệu và thuật toán cần thiết.

5.2 Tìm kiếm tuyến tính

Trong chương trình *Quản lý sinh viên*, thuật toán tìm kiếm tuyến tính được sử dụng để tìm kiếm dữ liệu trong các danh sách như sinh viên, môn học, lớp học phần và điểm số. Đây là thuật toán tìm kiếm cơ bản, phù hợp với dữ liệu được lưu trong mảng động và không yêu cầu danh sách phải được sắp xếp trước.

Ý tưởng của tìm kiếm tuyến tính là duyệt lần lượt từng phần tử trong danh sách, so sánh khóa của phần tử hiện tại với khóa cần tìm. Nếu tìm thấy phần tử có khóa trùng khớp, thuật toán trả về vị trí của phần tử đó trong mảng. Nếu duyệt hết danh sách mà không tìm thấy, thuật toán trả về giá trị báo hiệu không tồn tại, thường là `-1`.

Trong chương trình, thuật toán tìm kiếm tuyến tính được áp dụng cho nhiều loại dữ liệu khác nhau. Ví dụ, tìm sinh viên theo `MSSV`, tìm môn học theo `MaHP`, tìm lớp học phần theo `MaLHP`, hoặc tìm bản ghi điểm theo cặp `MSSV` và `MaLHP`.

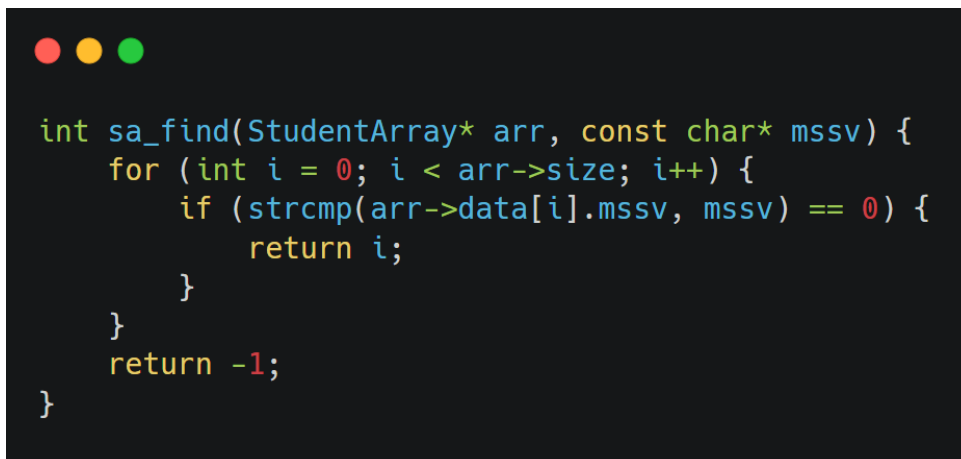
Mã giả thuật toán

```
LinearSearch(A, n, key)
    for i = 0 to n - 1 do
        if A[i].key == key then
            return i
    return -1
```

Trong đó:

- A: danh sách dữ liệu cần tìm kiếm.
- n: số lượng phần tử trong danh sách.
- key: khóa cần tìm.
- Giá trị trả về là vị trí phần tử nếu tìm thấy, hoặc -1 nếu không tìm thấy.

Ví dụ, hàm tìm kiếm sinh viên theo mã số sinh viên có thể được mô tả như sau:

A screenshot of a code editor with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The code is written in C and implements a linear search function. It takes a pointer to an array of students and a string of the student's ID as input. It iterates through the array, comparing each student's ID with the input string. If a match is found, it returns the index of that student. If no match is found after checking all students, it returns -1.

```
int sa_find(StudentArray* arr, const char* mssv) {
    for (int i = 0; i < arr->size; i++) {
        if (strcmp(arr->data[i].mssv, mssv) == 0) {
            return i;
        }
    }
    return -1;
}
```

Tương tự, các hàm tìm kiếm khác như `suba_find`, `cca_find` và `sca_find` cũng được xây dựng theo cùng nguyên tắc. Điểm khác nhau chủ yếu nằm ở trường dữ liệu được dùng làm khóa tìm kiếm.

Độ phức tạp thời gian của thuật toán tìm kiếm tuyến tính là $O(n)$ trong trường hợp xấu nhất, khi phần tử cần tìm nằm ở cuối danh sách hoặc không tồn tại trong danh sách. Độ phức tạp bộ nhớ là $O(1)$ vì thuật toán chỉ sử dụng một số biến phụ và không cần cấp phát thêm vùng nhớ đáng kể.

Mặc dù tìm kiếm tuyến tính không phải là thuật toán tối ưu nhất với dữ liệu lớn, nhưng thuật toán này đơn giản, dễ cài đặt và phù hợp với phạm vi của bài tập lớn. Ngoài

ra, vì danh sách dữ liệu trong chương trình không phải lúc nào cũng được sắp xếp, tìm kiếm tuyến tính là lựa chọn phù hợp để đảm bảo chương trình có thể tìm kiếm chính xác trong mọi trường hợp.

5.3 Sắp xếp sinh viên

Trong chương trình *Quản lý sinh viên*, chức năng sắp xếp được sử dụng để sắp xếp danh sách sinh viên theo một tiêu chí nhất định, giúp việc tra cứu và hiển thị dữ liệu trở nên thuận tiện hơn. Chương trình hỗ trợ sắp xếp sinh viên theo mã số sinh viên hoặc theo họ tên.

Chương trình hỗ trợ sắp xếp danh sách sinh viên theo ba tiêu chí: MSSV, họ tên và GPA. Việc sắp xếp theo MSSV và họ tên được thực hiện bằng cách so sánh chuỗi thông qua hàm `strcmp`. Đối với sắp xếp theo GPA, chương trình tính GPA hệ 10 của từng sinh viên, sau đó sắp xếp theo thứ tự giảm dần để sinh viên có GPA cao hơn đứng trước.

Thuật toán được sử dụng là Bubble Sort. Đây là thuật toán đơn giản, dễ cài đặt và phù hợp với quy mô dữ liệu nhỏ của bài tập lớn. Mặc dù Bubble Sort chưa tối ưu với dữ liệu lớn do độ phức tạp trung bình và xấu là $O(n^2)$, thuật toán này giúp thể hiện rõ quá trình so sánh và hoán đổi phần tử, phù hợp với mục tiêu rèn luyện kỹ thuật lập trình cơ bản.

Mã giả thuật toán

```
BubbleSort(A, n)
    for i = 0 to n - 2 do
        for j = 0 to n - i - 2 do
            if A[j] > A[j + 1] then
                swap(A[j], A[j + 1])
```

Trong chương trình, khi sắp xếp theo mã số sinh viên, khóa so sánh là trường `mssv`. Hai sinh viên liên tiếp sẽ được so sánh theo mã số sinh viên, nếu sinh viên đứng trước có mã lớn hơn sinh viên đứng sau thì chương trình thực hiện đổi chỗ.

```

if (strcmp(students->data[j].mssv,
          students->data[j + 1].mssv) > 0) {
    swap(students->data[j], students->data[j + 1]);
}

```

Tương tự, khi sắp xếp theo họ tên, chương trình sử dụng trường `hoTen` làm khóa so sánh. Việc so sánh chuỗi được thực hiện bằng hàm `strcmp`. Nếu tên của sinh viên đứng trước lớn hơn tên của sinh viên đứng sau theo thứ tự từ điển, hai phần tử sẽ được hoán đổi vị trí.

```

if (strcmp(students->data[j].hoTen,
          students->data[j + 1].hoTen) > 0) {
    swap(students->data[j], students->data[j + 1]);
}

```

Các thao tác sắp xếp được cài đặt trong module `sort.h/.c`. Module này cung cấp các hàm như `sortStudentByMSSV`, `sortStudentByName`, `sortStudentByGPA`. Khi người dùng chọn chức năng sắp xếp trong menu quản lý sinh viên, chương trình sẽ gọi hàm tương ứng và hiển thị lại danh sách sinh viên sau khi sắp xếp.

Độ phức tạp thời gian của Bubble Sort là $O(n^2)$ trong trường hợp trung bình và xấu nhất, với n là số lượng sinh viên trong danh sách. Độ phức tạp bộ nhớ là $O(1)$ vì thuật toán chỉ sử dụng một biến tạm để hoán đổi hai phần tử.

Mặc dù Bubble Sort không phải là thuật toán tối ưu cho dữ liệu lớn, nhưng thuật toán này có ưu điểm là đơn giản, dễ hiểu và dễ cài đặt. Với phạm vi của bài tập lớn và số lượng dữ liệu không quá lớn, Bubble Sort đáp ứng tốt yêu cầu tự cài đặt thuật toán sắp xếp và phù hợp để minh họa kỹ thuật xử lý dữ liệu trong chương trình.

5.4 Công thức tính điểm và GPA

Trong chương trình *Quản lý sinh viên*, điểm số của sinh viên được quản lý theo từng lớp học phần. Mỗi bản ghi điểm gồm điểm quá trình, điểm cuối kỳ, điểm tổng kết và điểm quy đổi hệ 4. Các công thức tính điểm được xây dựng nhằm hỗ trợ việc đánh giá kết quả học tập của sinh viên một cách rõ ràng và thống nhất.

Tính điểm tổng kết

Điểm tổng kết của một lớp học phần được tính dựa trên điểm quá trình và điểm cuối kỳ. Trong chương trình, hai thành phần điểm này được lấy với trọng số bằng nhau:

$$DiemTK = \frac{DiemQT + DiemCK}{2}$$

Trong đó:

- *DiemQT*: điểm quá trình của sinh viên.
- *DiemCK*: điểm cuối kỳ của sinh viên.
- *DiemTK*: điểm tổng kết của lớp học phần.

Ví dụ, nếu sinh viên có điểm quá trình là 8.0 và điểm cuối kỳ là 7.5, điểm tổng kết được tính như sau:

$$DiemTK = \frac{8.0 + 7.5}{2}$$

Quy đổi điểm hệ 4

Sau khi tính được điểm tổng kết, chương trình thực hiện quy đổi điểm từ thang điểm 10 sang thang điểm 4. Việc quy đổi này giúp chương trình có thể tính GPA hệ 4 cho từng sinh viên.

Công thức quy đổi được mô tả như sau:

$$DiemHe4 = \begin{cases} 4.0, & DiemTK \geq 8.5 \\ 3.5, & 8.0 \leq DiemTK < 8.5 \\ 3.0, & 7.0 \leq DiemTK < 8.0 \\ 2.5, & 6.5 \leq DiemTK < 7.0 \\ 2.0, & 5.5 \leq DiemTK < 6.5 \\ 1.5, & 5.0 \leq DiemTK < 5.5 \\ 1.0, & 4.0 \leq DiemTK < 5.0 \\ 0.0, & DiemTK < 4.0 \end{cases}$$

Việc quy đổi điểm hệ 4 được thực hiện tự động sau khi chương trình nhập hoặc cập nhật điểm quá trình và điểm cuối kỳ.

Tính GPA hệ 10

GPA hệ 10 của một sinh viên được tính dựa trên điểm tổng kết của các lớp học phần mà sinh viên đó đã có điểm. Vì mỗi môn học có số tín chỉ khác nhau, chương trình cần tính trung bình có trọng số theo số tín chỉ.

Công thức tính GPA hệ 10:

$$GPA_{10} = \frac{\sum_{i=1}^n DiemTK_i \times SoTinChi_i}{\sum_{i=1}^n SoTinChi_i}$$

Trong đó:

- $DiemTK_i$: điểm tổng kết của học phần thứ i .
- $SoTinChi_i$: số tín chỉ của học phần thứ i .
- n : số học phần sinh viên đã có điểm.

Tính GPA hệ 4

Tương tự GPA hệ 10, GPA hệ 4 được tính dựa trên điểm hệ 4 của từng học phần và số tín chỉ tương ứng:

$$GPA_4 = \frac{\sum_{i=1}^n DiemHe4_i \times SoTinChi_i}{\sum_{i=1}^n SoTinChi_i}$$

Trong chương trình, để tính GPA, cần liên kết dữ liệu giữa bản ghi điểm, lớp học phần và môn học. Cụ thể, từ `ScoreRecord` lấy được `MaLHP`, từ `CourseClass` xác định `MaHP`, sau đó từ `Subject` lấy số tín chỉ của môn học tương ứng.

Nhìn chung, các công thức trên giúp chương trình không chỉ lưu trữ điểm số mà còn có khả năng xử lý và tổng hợp kết quả học tập của sinh viên. Đây là một phần quan trọng trong đề tài, thể hiện việc vận dụng kiến thức lập trình, cấu trúc dữ liệu và xử lý dữ liệu có liên kết.

Chương 6

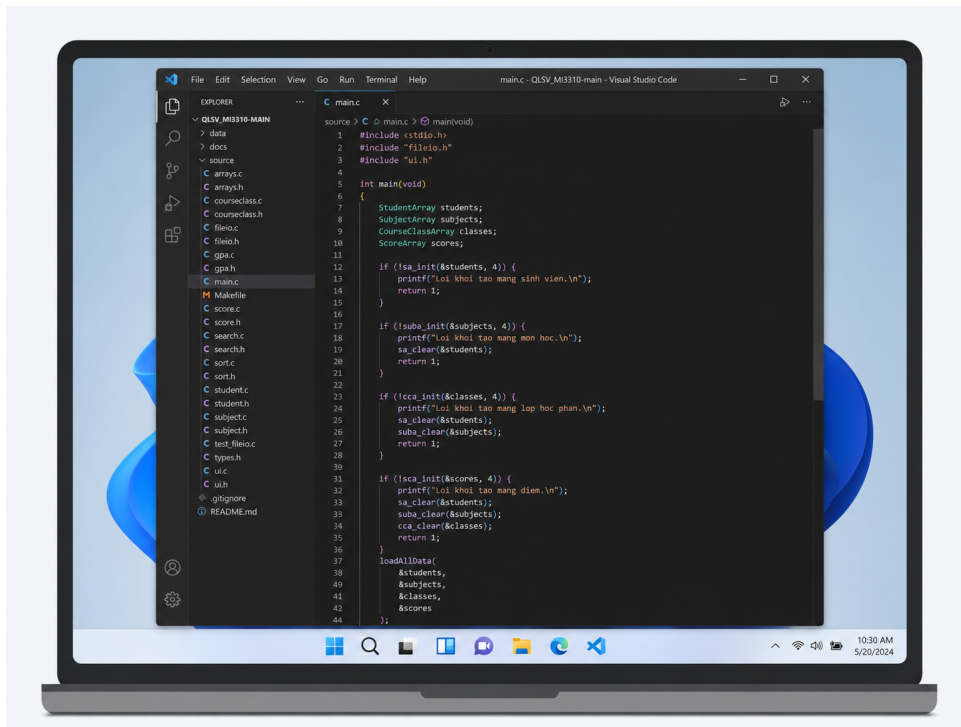
CÀI ĐẶT VÀ KIỂM THỬ

6.1 Môi trường cài đặt

Chương trình *Quản lý sinh viên* được xây dựng bằng ngôn ngữ lập trình C và chạy trên giao diện console. Trong quá trình phát triển, nhóm sử dụng trình biên dịch GCC để biên dịch mã nguồn, kết hợp với Makefile nhằm hỗ trợ quá trình build chương trình nhanh chóng và thuận tiện hơn.

Môi trường cài đặt và chạy chương trình được sử dụng như sau:

- **Hệ điều hành:** Windows 11.
- **Ngôn ngữ lập trình:** C.
- **Trình biên dịch:** GCC hoặc trình biên dịch C tương đương.
- **Công cụ build:** Makefile.
- **Trình soạn thảo mã nguồn:** Visual Studio Code.
- **Quản lý mã nguồn:** Git và GitHub.
- **Lưu trữ dữ liệu:** File text trong thư mục `data/`.



Hình 6.1: Ảnh minh họa cài đặt

```

1 cd source
2 make clean
3 make all
4 cd ..
5 ./qlsv.exe

```

Listing 6.1: Các lệnh build và chạy chương trình

```

1 cd source
2 make clean
3 make all
4 cd ..
5 .\qlsv.exe

```

Listing 6.2: Chạy chương trình trên Windows

Khi sử dụng Makefile, chương trình có thể được biên dịch bằng lệnh:

```

cd source
make all

```

Trong trường hợp không sử dụng Makefile, có thể biên dịch thủ công bằng GCC bằng cách liên kết các file mã nguồn cần thiết:

```

gcc -Wall -Wextra -std=c99 arrays.c fileio.c student.c subject.c courseclass.c
score.c gpa.c sort.c search.c ui.c main.c -o ../qlsv.exe

```

Sau khi biên dịch, chương trình cần được chạy từ thư mục gốc của dự án để các đường dẫn đến file dữ liệu trong thư mục `data/` hoạt động chính xác. Trên Windows, chương trình có thể được chạy bằng file thực thi `qlsv.exe`.

Việc sử dụng môi trường cài đặt trên giúp nhóm dễ dàng phát triển, kiểm thử và đóng gói chương trình. Đồng thời, các công cụ như GCC, Makefile và Git cũng hỗ trợ quá trình quản lý mã nguồn, phát hiện lỗi biên dịch và kiểm tra chương trình trong quá trình thực hiện bài tập lớn.

6.2 Các chức năng đã cài đặt

Dựa trên yêu cầu của đề tài, chương trình *Quản lý sinh viên* đã cài đặt các nhóm chức năng chính phục vụ việc quản lý dữ liệu sinh viên, môn học, lớp học phần, điểm số và báo cáo bảng điểm. Các chức năng được tổ chức thông qua hệ thống menu console, cho phép người dùng lựa chọn thao tác cần thực hiện.

Chức năng quản lý sinh viên

Nhóm chức năng quản lý sinh viên cho phép người dùng thao tác với danh sách sinh viên trong chương trình. Các chức năng đã cài đặt gồm:

- Hiển thị danh sách sinh viên.
- Thêm sinh viên mới.
- Sửa thông tin sinh viên.
- Xóa sinh viên.
- Tìm kiếm sinh viên theo mã số sinh viên, họ tên hoặc lớp.
- Sắp xếp danh sách sinh viên theo mã số sinh viên hoặc họ tên.

Khi thêm sinh viên, chương trình kiểm tra mã số sinh viên để tránh trùng lặp. Khi xóa sinh viên, chương trình kiểm tra xem sinh viên đó đã có bản ghi điểm hay chưa. Nếu sinh viên đã có điểm, chương trình không cho phép xóa nhằm đảm bảo tính nhất quán dữ liệu.

Chức năng quản lý môn học

Chương trình hỗ trợ quản lý danh sách môn học với các thao tác cơ bản như:

- Hiển thị danh sách môn học.

- Thêm môn học mới.
- Sửa thông tin môn học.
- Xóa môn học.
- Tìm kiếm môn học theo mã học phần hoặc tên học phần.

Mỗi môn học được quản lý thông qua mã học phần, tên học phần và số tín chỉ. Khi xóa môn học, chương trình kiểm tra xem môn học đó có đang được sử dụng bởi lớp học phần nào hay không. Nếu vẫn còn lớp học phần sử dụng mã học phần đó, thao tác xóa sẽ bị hủy.

Chức năng quản lý lớp học phần

Đối với lớp học phần, chương trình đã cài đặt các chức năng:

- Hiện thị danh sách lớp học phần.
- Thêm lớp học phần mới.
- Sửa thông tin lớp học phần.
- Xóa lớp học phần.
- Tìm kiếm lớp học phần theo mã lớp học phần hoặc mã học phần.

Mỗi lớp học phần được liên kết với một môn học thông qua mã học phần. Khi thêm hoặc sửa lớp học phần, chương trình kiểm tra mã học phần có tồn tại trong danh sách môn học hay không. Khi xóa lớp học phần, chương trình kiểm tra xem lớp đó đã có bản ghi điểm hay chưa để tránh làm mất liên kết dữ liệu.

Chức năng quản lý điểm số

Nhóm chức năng quản lý điểm số cho phép nhập và cập nhật kết quả học tập của sinh viên theo từng lớp học phần. Các chức năng đã cài đặt gồm:

- Hiện thị danh sách điểm.
- Nhập điểm cho sinh viên.
- Cập nhật điểm đã có.
- Tìm kiếm điểm theo mã số sinh viên, mã lớp học phần hoặc cặp mã số sinh viên và mã lớp học phần.

Khi nhập điểm, chương trình kiểm tra sinh viên và lớp học phần có tồn tại hay không. Ngoài ra, điểm quá trình và điểm cuối kỳ phải nằm trong khoảng từ 0 đến 10. Sau khi nhập hoặc cập nhật điểm, chương trình tự động tính điểm tổng kết và điểm quy đổi hệ 4.

Chức năng báo cáo và hiển thị bảng điểm

Chương trình cũng hỗ trợ một số chức năng báo cáo cơ bản, bao gồm:

- Hiển thị bảng điểm của một sinh viên.
- Hiển thị bảng điểm của một lớp học phần.
- Hiển thị toàn bộ dữ liệu hiện có trong chương trình.

Các chức năng báo cáo giúp người dùng dễ dàng theo dõi kết quả học tập của sinh viên và tình hình điểm số trong từng lớp học phần.

Chức năng đọc, ghi file và kiểm tra dữ liệu

Bên cạnh các chức năng quản lý chính, chương trình còn cài đặt cơ chế đọc và ghi dữ liệu bằng file text. Khi chương trình khởi động, dữ liệu được đọc từ các file trong thư mục `data/`. Khi người dùng chọn lưu và thoát, dữ liệu hiện tại trong bộ nhớ sẽ được ghi lại xuống file.

Ngoài ra, chương trình có các hàm kiểm tra dữ liệu đầu vào như kiểm tra mã sinh viên, ngày sinh, điểm số, số tín chỉ, học kỳ và năm học. Việc kiểm tra dữ liệu giúp hạn chế lỗi nhập liệu và tăng tính ổn định cho chương trình.

Nhìn chung, các chức năng đã cài đặt đáp ứng được yêu cầu chính của đề tài *Quản lý sinh viên*. Chương trình cho phép quản lý dữ liệu cơ bản, xử lý điểm số, tìm kiếm, sắp xếp, hiển thị báo cáo và lưu trữ dữ liệu bằng file text.

6.3 Dữ liệu kiểm thử

Dữ liệu kiểm thử được xây dựng nhằm kiểm tra khả năng hoạt động của chương trình trong các chức năng chính như đọc dữ liệu, hiển thị danh sách, thêm/sửa/xóa thông tin, nhập điểm, tính điểm tổng kết, tính GPA và kiểm tra toàn vẹn dữ liệu. Toàn bộ dữ liệu được lưu trong thư mục `data/` dưới dạng file text, trong đó mỗi dòng biểu diễn một bản ghi và các trường dữ liệu được phân tách bằng ký tự `|`.

Chương trình sử dụng bốn file dữ liệu chính: `students.txt`, `subjects.txt`, `course_classes.txt` và `scores.txt`. Các file này tương ứng với bốn nhóm dữ liệu quan trọng trong hệ thống là sinh viên, môn học, lớp học phần và điểm số.

```
1 cd source
2 make unit_test
3 make test
```

Listing 6.3: Các lệnh chạy kiểm thử

Bảng 6.1: Các file dữ liệu kiểm thử

File dữ liệu	Nội dung lưu trữ	Vai trò trong kiểm thử
<code>students.txt</code>	Thông tin sinh viên gồm MSSV, họ tên, lớp và ngày sinh	Kiểm tra chức năng quản lý sinh viên, tìm kiếm sinh viên và liên kết với bảng điểm
<code>subjects.txt</code>	Thông tin môn học gồm mã học phần, tên học phần và số tín chỉ	Kiểm tra chức năng quản lý môn học và phục vụ tính GPA theo tín chỉ
<code>course_classes.txt</code>	Thông tin lớp học phần gồm mã lớp học phần, mã học phần, học kỳ và năm học	Kiểm tra quan hệ giữa lớp học phần và môn học
<code>scores.txt</code>	Điểm của sinh viên theo lớp học phần, gồm điểm quá trình, điểm cuối kỳ, điểm tổng kết và điểm hệ 4	Kiểm tra chức năng nhập điểm, cập nhật điểm, tính điểm tổng kết, tính GPA và in bảng điểm

Trong bộ dữ liệu kiểm thử chính, chương trình sử dụng dữ liệu mẫu gồm 6 sinh viên, 5 môn học, 5 lớp học phần và 22 bản ghi điểm. Quy mô dữ liệu này đủ để kiểm tra các chức năng cơ bản của chương trình, đồng thời vẫn đảm bảo dễ quan sát kết quả khi chạy trên giao diện console.

Bảng 6.2: Quy mô dữ liệu kiểm thử chính

Nhóm dữ liệu	Số lượng bản ghi	Mục đích kiểm thử
Sinh viên	6	Kiểm tra hiển thị danh sách, tìm kiếm, sắp xếp và liên kết với điểm số
Môn học	5	Kiểm tra quản lý môn học và số tín chỉ khi tính GPA
Lớp học phần	5	Kiểm tra quan hệ giữa môn học và lớp học phần
Bản ghi điểm	22	Kiểm tra nhập điểm, cập nhật điểm, tính điểm tổng kết, quy đổi hệ 4 và tính GPA

Một số dòng dữ liệu mẫu trong file `students.txt` có dạng như sau:

```
MSSV|HoTen|Lop|Birthday
202400000|Nguyen Van Toan|K69-MI1-01|15/08/2006
202400001|Tran Quan Anh|K69-MI1-02|20/03/2006
...
```

Một số dòng dữ liệu mẫu trong file `subjects.txt`:

```
MaHP|TenHP|SoTinChi
MI3310|Ky Thuat Lap Trinh|2
MI3060|Cau Truc Du Lieu & Thuat Toan|3
...
```

Một số dòng dữ liệu mẫu trong file `course_classes.txt`:

```
MaLHP|MaHP|HocKy|NamHoc
169313|MI3310|1|2025
169307|MI3060|2|2025
...
```

Một số dòng dữ liệu mẫu trong file `scores.txt`:

```
MSSV|MaLHP|DiemQT|DiemCK|DiemTK|DiemHe4
202400000|169313|8.50|7.00|7.75|3.00
202400000|169307|8.00|9.00|8.50|4.00
202400000|169320|6.50|7.00|6.75|2.50
202400000|169400|9.00|8.00|8.50|4.00
202400000|169401|7.50|8.00|7.75|3.00
...
```

Ngoài dữ liệu hợp lệ, nhóm cũng chuẩn bị một số tình huống dữ liệu lỗi để kiểm tra khả năng xử lý ngoại lệ của chương trình. Các tình huống này bao gồm file không tồn tại, file rỗng, dòng thiếu trường, khóa chính bị trùng, điểm nằm ngoài khoảng từ 0 đến 10 và dữ liệu tham chiếu không tồn tại. Khi gặp các dòng dữ liệu không hợp lệ, chương trình sẽ bỏ qua dòng lỗi, hiển thị cảnh báo và tiếp tục nạp các dữ liệu hợp lệ còn lại.

Các nhóm dữ liệu kiểm thử được sử dụng gồm:

- **Dữ liệu hợp lệ:** dùng để kiểm tra các chức năng chính như hiển thị danh sách, tìm kiếm, sắp xếp, nhập điểm và in bảng điểm.
- **Dữ liệu trùng khóa:** dùng để kiểm tra việc không cho phép thêm sinh viên, môn học, lớp học phần hoặc bản ghi điểm bị trùng khóa.
- **Dữ liệu sai định dạng:** dùng để kiểm tra khả năng bỏ qua dòng thiếu trường hoặc trường bắt buộc bị rỗng.
- **Dữ liệu điểm không hợp lệ:** dùng để kiểm tra việc từ chối điểm nhỏ hơn 0 hoặc lớn hơn 10.

- **Dữ liệu sai tham chiếu:** dùng để kiểm tra các trường hợp mã sinh viên, mã học phần hoặc mã lớp học phần không tồn tại.
- **Dữ liệu biên:** dùng để kiểm tra thao tác với mảng rỗng, chỉ số không hợp lệ và file không tồn tại.

Nhờ có dữ liệu kiểm thử đa dạng, chương trình có thể được kiểm tra ở cả hai mức: kiểm thử trực tiếp qua giao diện console và kiểm thử tự động bằng các file test riêng. Điều này giúp đánh giá tốt hơn độ ổn định của các module nền tảng như mảng động, đọc/ghi file, tính điểm và tính GPA.

6.4 Bảng test case

Để đánh giá độ đúng đắn và độ ổn định của chương trình, nhóm xây dựng các test case bao phủ các chức năng chính như quản lý sinh viên, quản lý môn học, quản lý lớp học phần, quản lý điểm số, tìm kiếm, sắp xếp, tính GPA, đọc/ghi file và kiểm tra dữ liệu không hợp lệ. Các test case được thực hiện thông qua giao diện console kết hợp với các file kiểm thử riêng cho module nền tảng.

Quá trình kiểm thử của nhóm được xây dựng theo hướng kiểm tra từng module quan trọng trước, sau đó kiểm tra tích hợp toàn bộ quy trình đọc/ghi dữ liệu. Cách làm này phù hợp với nguyên tắc viết từng phần, kiểm tra từng phần và phát hiện lỗi sớm trong quá trình lập trình.

Các unit test như `test_types.c`, `test_arrays.c`, `test_fileio_unit.c` và `test_gpa.c` được dùng để kiểm tra riêng từng thành phần nhỏ. Sau đó, `test_fileio.c` được dùng để kiểm tra tích hợp với dữ liệu thật trong thư mục `data/`. Nhờ đó, nhóm có thể kiểm tra cả tính đúng đắn của cấu trúc dữ liệu, xử lý file, công thức tính GPA và toàn vẹn khóa ngoại.

Bên cạnh kiểm thử dữ liệu hợp lệ, nhóm cũng chủ động đưa vào các trường hợp sai như file không tồn tại, dòng thiếu trường, mã sinh viên rỗng, dữ liệu trùng khóa và khóa ngoại không hợp lệ. Đây là cách áp dụng tư tưởng lập trình phòng ngừa, giúp chương trình không bị dừng đột ngột khi gặp dữ liệu lỗi mà có thể bỏ qua bản ghi sai và tiếp tục xử lý các dữ liệu hợp lệ còn lại.

Bảng 6.3: Bảng test case chức năng của chương trình

Mã TC	Chức năng	Dữ liệu / tình huống kiểm thử	Kết quả mong đợi	Kết quả
TC01	Thêm sinh viên hợp lệ	Nhập sinh viên mới với MSSV chưa tồn tại	Sinh viên được thêm vào danh sách	Đạt
TC02	Thêm sinh viên trùng MSSV	Nhập MSSV đã tồn tại trong danh sách sinh viên	Chương trình hiển thị lỗi và không thêm dữ liệu	Đạt
TC03	Cập nhật sinh viên	Nhập MSSV tồn tại, sau đó sửa họ tên, lớp hoặc ngày sinh	Thông tin sinh viên được cập nhật chính xác	Đạt
TC04	Xóa sinh viên chưa có điểm	Xóa sinh viên không xuất hiện trong file điểm	Sinh viên được xóa khỏi danh sách	Đạt
TC05	Xóa sinh viên đã có điểm	Xóa sinh viên có MSSV đang tồn tại trong <code>scores.txt</code>	Chương trình thông báo lỗi và không cho xóa	Đạt
TC06	Thêm môn học hợp lệ	Nhập môn học mới với mã học phần chưa tồn tại	Môn học được thêm thành công	Đạt
TC07	Thêm môn học trùng mã	Nhập mã học phần đã tồn tại	Chương trình báo lỗi và không thêm môn học	Đạt
TC08	Xóa môn học đang được sử dụng	Xóa môn học có mã học phần đang được lớp học phần tham chiếu	Chương trình không cho xóa để đảm bảo toàn vẹn dữ liệu	Đạt
TC09	Thêm lớp học phần hợp lệ	Nhập mã lớp học phần mới và mã học phần đã tồn tại	Lớp học phần được thêm thành công	Đạt
TC10	Thêm lớp học phần sai mã học phần	Nhập mã học phần không tồn tại trong danh sách môn học	Chương trình báo lỗi và không thêm lớp học phần	Đạt
TC11	Xóa lớp học phần đã có điểm	Xóa lớp học phần có mã đang tồn tại trong <code>scores.txt</code>	Chương trình không cho xóa lớp học phần	Đạt
TC12	Nhập điểm hợp lệ	Nhập MSSV, mã lớp học phần, điểm quá trình và điểm cuối kỳ hợp lệ	Bản ghi điểm được thêm; điểm tổng kết và điểm hệ 4 được tính tự động	Đạt

Mã TC	Chức năng	Dữ liệu / tình huống kiểm thử	Kết quả mong đợi	Kết quả
TC13	Nhập điểm ngoài khoảng	Nhập điểm quá trình hoặc điểm cuối kỳ nhỏ hơn 0 hoặc lớn hơn 10	Chương trình báo lỗi và yêu cầu nhập lại	Đạt
TC14	Nhập điểm cho sinh viên không tồn tại	Nhập MSSV không có trong danh sách sinh viên	Chương trình báo lỗi và không lưu điểm	Đạt
TC15	Nhập điểm trùng	Nhập điểm cho cùng một cặp MSSV và mã lớp học phần đã tồn tại	Chương trình không tạo bản ghi trùng	Đạt
TC16	Cập nhật điểm	Cập nhật điểm quá trình hoặc điểm cuối kỳ của bản ghi đã tồn tại	Điểm được cập nhật, điểm tổng kết và điểm hệ 4 được tính lại	Đạt
TC17	Tính điểm tổng kết	Điểm quá trình là 8.5, điểm cuối kỳ là 7.0	Điểm tổng kết bằng 7.75 theo công thức trung bình có trọng số	Đạt
TC18	Tính GPA	Sinh viên có nhiều bản ghi điểm thuộc các môn có số tín chỉ khác nhau	GPA được tính theo công thức trung bình có trọng số tín chỉ	Đạt
TC19	Tìm kiếm sinh viên	Tìm sinh viên theo MSSV hoặc họ tên đã tồn tại	Chương trình hiển thị đúng thông tin sinh viên	Đạt
TC20	Tìm kiếm sinh viên không tồn tại	Nhập MSSV không có trong danh sách	Chương trình thông báo không tìm thấy	Đạt
TC21	Sắp xếp sinh viên theo MSSV	Danh sách sinh viên chưa được sắp xếp	Danh sách được hiển thị theo thứ tự tăng dần của MSSV	Đạt
TC22	Sắp xếp sinh viên theo họ tên	Danh sách sinh viên chưa được sắp xếp theo tên	Danh sách được hiển thị theo thứ tự họ tên	Đạt
TC23	Sắp xếp sinh viên theo GPA	Danh sách sinh viên chưa được sắp xếp theo GPA	Danh sách được hiển thị theo thứ tự theo thứ tự giảm dần của GPA	Đạt
TC24	Hiển thị bảng điểm sinh viên	Nhập MSSV của sinh viên đã có điểm	Chương trình hiển thị các lớp học phần, điểm thành phần, điểm tổng kết, điểm hệ 4 và GPA	Đạt

Mã TC	Chức năng	Dữ liệu / tình huống kiểm thử	Kết quả mong đợi	Kết quả
TC25	Hiển thị bảng điểm lớp học phần	Nhập mã lớp học phần có nhiều sinh viên đã nhập điểm	Chương trình hiển thị danh sách sinh viên và điểm trong lớp học phần đó	Đạt
TC26	Đọc file không tồn tại	Truyền đường dẫn file dữ liệu không tồn tại	Chương trình không bị crash và tạo danh sách rỗng	Đạt
TC27	Đọc file sai định dạng	File có dòng thiếu trường, khóa rỗng hoặc dữ liệu không hợp lệ	Chương trình bỏ qua dòng lỗi và tiếp tục nạp dữ liệu hợp lệ	Đạt
TC28	Lưu dữ liệu và nạp lại	Thêm hoặc sửa dữ liệu, thoát chương trình, sau đó chạy lại	Dữ liệu đã thay đổi vẫn được lưu và nạp lại chính xác	Đạt
TC29	Quy đổi điểm hệ 4	Điểm tổng kết thuộc các mốc quy đổi khác nhau	Chương trình quy đổi đúng điểm hệ 4 theo khoảng điểm đã cài đặt	Đạt

Ngoài các test case chức năng, nhóm còn thực hiện kiểm thử tự động cho các module nền tảng. Các file kiểm thử gồm `test_types.c`, `test_arrays.c`, `test_fileio_unit.c`, `test_gpa.c` và `test_fileio.c`. Kết quả tổng hợp cho thấy toàn bộ 68 test case của các module nền tảng đều đạt yêu cầu.

Bảng 6.4: Kết quả kiểm thử tự động các module nền tảng

File kiểm thử	Loại kiểm thử	Số test case	Kết quả
<code>test_types.c</code>	Unit test	7	7/7 PASS
<code>test_arrays.c</code>	Unit test	11	11/11 PASS
<code>test_fileio_unit.c</code>	Unit test	6	6/6 PASS
<code>test_gpa.c</code>	Unit test	3	3/3 PASS
<code>test_fileio.c</code>	Integration test	41	41/41 PASS
Tổng cộng		68	68/68 PASS

Một số test case có xuất hiện cảnh báo trong quá trình chạy, ví dụ khi đọc file sai định dạng hoặc gặp dữ liệu không hợp lệ. Các cảnh báo này là tình huống được tạo có chủ đích để kiểm tra khả năng xử lý lỗi của chương trình, không phải lỗi thực tế. Kết quả kiểm thử cho thấy chương trình xử lý tốt các trường hợp dữ liệu hợp lệ, dữ liệu trùng khóa, dữ liệu sai định dạng, dữ liệu sai tham chiếu và các thao tác cơ bản trên mảng động.

Chương 7

KẾT QUẢ THỰC HIỆN

7.1 Kiến trúc chương trình

Hệ thống được xây dựng theo mô hình phân tách module, gồm các thành phần chính:

- Module quản lý sinh viên.
- Module quản lý môn học.
- Module quản lý lớp học phần.
- Module quản lý điểm số.
- Module đọc/ghi file.
- Module giao diện console.

Hàm `main()` có nhiệm vụ khởi tạo dữ liệu, nạp dữ liệu từ file, gọi menu chính và lưu dữ liệu khi kết thúc chương trình.

```
1 loadAllData(...)  
2 showMainMenu(...)  
3 saveAllData(...)
```

Mã nguồn đầy đủ được trình bày trong Phụ lục A.

7.2 Module quản lý sinh viên

Module sinh viên quản lý thông tin gồm MSSV, họ tên, lớp và ngày sinh. Các chức năng chính:

- Thêm sinh viên.
- Cập nhật thông tin sinh viên.

- Xóa sinh viên.
- Tìm kiếm sinh viên.
- Sắp xếp theo MSSV, họ tên hoặc GPA.

Module sử dụng MSSV làm khóa chính để đảm bảo tính duy nhất của dữ liệu.

Bảng 7.1: Chức năng của module sinh viên

Hàm	Chức năng
findStudentRecord	Tìm sinh viên
addStudentRecord	Thêm sinh viên
updateStudentRecord	Cập nhật sinh viên
deleteStudentRecord	Xóa sinh viên

7.3 Module quản lý môn học

Module môn học quản lý mã học phần, tên học phần và số tín chỉ.

Các chức năng chính:

- Thêm môn học.
- Cập nhật môn học.
- Xóa môn học.
- Tìm kiếm môn học.

Mã học phần được sử dụng làm khóa chính.

7.4 Module quản lý lớp học phần

Module lớp học phần quản lý các lớp học phần được mở theo từng học kỳ.

Thông tin quản lý gồm:

- Mã lớp học phần.
- Mã học phần.
- Học kỳ.
- Năm học.

Hệ thống kiểm tra tính hợp lệ của mã học phần trước khi tạo lớp học phần mới nhằm đảm bảo liên kết dữ liệu.

7.5 Module quản lý điểm số

Module điểm số quản lý kết quả học tập của sinh viên.

Các chức năng chính:

- Nhập điểm.
- Cập nhật điểm.
- Tính điểm tổng kết.
- Quy đổi điểm hệ 4.

Điểm tổng kết được tính theo công thức:

$$DiemTK = \frac{DiemQT + DiemCK}{2}$$

Sau khi tính điểm tổng kết, hệ thống tự động quy đổi sang thang điểm hệ 4 để phục vụ tính GPA.

7.6 Module đọc/ghi file

Module đọc/ghi file chịu trách nhiệm lưu trữ và phục hồi dữ liệu.

Dữ liệu được lưu thành bốn tệp:

- students.txt
- subjects.txt
- course_classes.txt
- scores.txt

Mỗi loại dữ liệu được lưu trong một tệp riêng nhằm tăng tính rõ ràng và thuận tiện cho việc bảo trì.

7.7 Module giao diện console

Giao diện console cho phép người dùng thao tác với toàn bộ chức năng của hệ thống thông qua menu.

Các nhóm chức năng:

- Quản lý sinh viên.

- Quản lý môn học.
- Quản lý lớp học phần.
- Quản lý điểm số.
- Báo cáo kết quả học tập.

Dữ liệu nhập từ người dùng được kiểm tra trước khi chuyển đến các module nghiệp vụ nhằm đảm bảo tính hợp lệ.

Chương 8

KẾT LUẬN

8.1 Kết quả đạt được

Sau quá trình phân tích, thiết kế, cài đặt và kiểm thử, nhóm đã xây dựng được chương trình *Quản lý sinh viên* chạy trên giao diện console bằng ngôn ngữ lập trình C. Chương trình đáp ứng được các yêu cầu chính của đề tài, bao gồm quản lý sinh viên, môn học, lớp học phần, điểm số, tìm kiếm, sắp xếp, tính GPA và lưu trữ dữ liệu bằng file text.

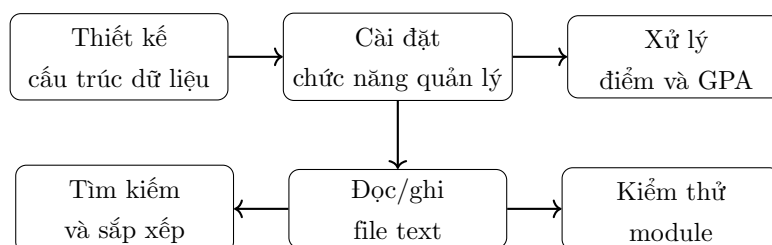
Trước hết, nhóm đã thiết kế được hệ thống dữ liệu tương đối rõ ràng thông qua các **struct** chính như **Student**, **Subject**, **CourseClass** và **ScoreRecord**. Các kiểu dữ liệu này giúp biểu diễn các đối tượng cần quản lý trong chương trình một cách trực tiếp và dễ hiểu. Bên cạnh đó, nhóm cũng tự cài đặt cấu trúc dữ liệu mảng động cho từng nhóm dữ liệu, bao gồm **StudentArray**, **SubjectArray**, **CourseClassArray** và **ScoreArray**. Nhờ đó, chương trình có thể thêm, sửa, xóa, tìm kiếm và quản lý dữ liệu linh hoạt hơn so với việc sử dụng mảng tĩnh.

Về chức năng quản lý, chương trình đã cài đặt được các thao tác cơ bản đối với sinh viên, môn học và lớp học phần như thêm mới, cập nhật, xóa, tìm kiếm và hiển thị danh sách. Các thao tác này được tổ chức thành từng module riêng, giúp mã nguồn rõ ràng và dễ bảo trì hơn. Khi thêm dữ liệu mới, chương trình có kiểm tra khóa chính để hạn chế tình trạng trùng lặp, ví dụ như trùng mã số sinh viên, mã học phần hoặc mã lớp học phần.

Đối với phần điểm số, chương trình đã cài đặt chức năng nhập và cập nhật điểm cho sinh viên theo từng lớp học phần. Sau khi người dùng nhập điểm quá trình và điểm cuối kỳ, chương trình tự động tính điểm tổng kết và quy đổi sang điểm hệ 4. Ngoài ra, chương trình cũng hỗ trợ tính GPA của sinh viên dựa trên điểm hệ 4 và số tín chỉ của các học phần tương ứng.

Chương trình cũng đã cài đặt được các thuật toán cơ bản theo yêu cầu của bài tập lớn. Cụ thể, nhóm sử dụng tìm kiếm tuyến tính để tìm kiếm dữ liệu theo khóa và tự cài đặt thuật toán Bubble Sort để sắp xếp danh sách sinh viên theo mã số sinh viên hoặc họ tên. Các thuật toán này tuy đơn giản nhưng phù hợp với phạm vi đề tài và giúp nhóm vận dụng được kiến thức về cấu trúc dữ liệu và giải thuật trong ngôn ngữ C.

Về lưu trữ dữ liệu, nhóm đã xây dựng cơ chế đọc và ghi dữ liệu thông qua các file text trong thư mục `data/`. Khi chương trình khởi động, dữ liệu được nạp từ các file như `students.txt`, `subjects.txt`, `course_classes.txt` và `scores.txt`. Khi người dùng chọn lưu và thoát, dữ liệu trong bộ nhớ được ghi lại xuống file. Cơ chế này giúp dữ liệu được lưu trữ lâu dài và có thể sử dụng lại ở các lần chạy sau.



Bên cạnh việc cài đặt chức năng, nhóm cũng đã xây dựng một số file kiểm thử cho các module quan trọng như mảng động, đọc/ghi file, tính GPA và định nghĩa kiểu dữ liệu. Các file kiểm thử này giúp kiểm tra các trường hợp cơ bản, trường hợp biên, dữ liệu sai định dạng, file không tồn tại và toàn vẹn khóa ngoại. Nhờ đó, nhóm có thể phát hiện và sửa lỗi trong quá trình hoàn thiện chương trình.

Thông qua đề tài, nhóm đã vận dụng được nhiều nội dung lý thuyết của học phần Kỹ thuật lập trình vào một chương trình cụ thể. Chương trình sử dụng cấu trúc dữ liệu tự cài đặt để lưu trữ dữ liệu, áp dụng thuật toán tìm kiếm tuyến tính và Bubble Sort để xử lý danh sách, tổ chức mã nguồn theo hướng module hóa và có các bước kiểm tra dữ liệu đầu vào nhằm phòng ngừa lỗi.

Ngoài ra, nhóm đã xây dựng các bộ kiểm thử để kiểm tra từng phần của chương trình cũng như kiểm tra tích hợp quá trình đọc/ghi dữ liệu. Kết quả kiểm thử cho thấy các module nền tảng hoạt động đúng với dữ liệu hợp lệ và xử lý được nhiều trường hợp dữ liệu sai định dạng. Đây là cơ sở quan trọng để đánh giá chương trình không chỉ chạy được mà còn có tính ổn định và dễ bảo trì.

Nhìn chung, chương trình đã đạt được mục tiêu ban đầu của đề tài. Sản phẩm cuối cùng có cấu trúc rõ ràng, dữ liệu được tổ chức hợp lý, các chức năng chính hoạt động được và có khả năng lưu trữ dữ liệu bằng file text. Thông qua quá trình thực hiện, nhóm

đã vận dụng được nhiều kiến thức quan trọng của học phần như thiết kế chương trình theo module, sử dụng struct, quản lý bộ nhớ động, đọc/ghi file, tìm kiếm, sắp xếp, kiểm tra dữ liệu đầu vào và kiểm thử chương trình.

8.2 Hạn chế

Mặc dù chương trình *Quản lý sinh viên* đã đáp ứng được các yêu cầu cơ bản của đề tài, trong quá trình thực hiện vẫn còn tồn tại một số hạn chế nhất định. Các hạn chế này chủ yếu xuất phát từ phạm vi của bài tập lớn, thời gian thực hiện và việc chương trình được xây dựng ở mức console đơn giản.

Trước hết, giao diện của chương trình mới dừng lại ở dạng console. Người dùng thao tác bằng cách nhập số và nhập dữ liệu từ bàn phím, chưa có giao diện đồ họa trực quan. Vì vậy, trải nghiệm sử dụng chưa thật sự thuận tiện nếu so sánh với các phần mềm quản lý hoàn chỉnh. Việc hiển thị dữ liệu trên console cũng còn đơn giản, đặc biệt khi danh sách sinh viên, môn học hoặc điểm số có nhiều bản ghi.

Thứ hai, dữ liệu của chương trình được lưu trữ bằng file text. Cách lưu trữ này dễ hiểu, dễ kiểm tra và phù hợp với yêu cầu của bài tập lớn, tuy nhiên vẫn còn một số hạn chế. Khi dữ liệu tăng lên nhiều, việc đọc ghi toàn bộ file có thể chưa tối ưu. Ngoài ra, file text cũng chưa có cơ chế bảo mật, phân quyền hoặc kiểm soát truy cập như khi sử dụng cơ sở dữ liệu thực tế.

Thứ ba, các thuật toán tìm kiếm và sắp xếp trong chương trình còn ở mức cơ bản. Chương trình sử dụng tìm kiếm tuyến tính và thuật toán Bubble Sort để xử lý dữ liệu. Các thuật toán này dễ cài đặt và phù hợp với quy mô nhỏ, nhưng chưa thật sự hiệu quả khi số lượng bản ghi lớn. Ví dụ, Bubble Sort có độ phức tạp thời gian $O(n^2)$ nên tốc độ xử lý sẽ giảm khi danh sách sinh viên tăng lên nhiều.

Thứ tư, chức năng tính điểm và GPA mới được cài đặt ở mức cơ bản. Chương trình đã hỗ trợ tính điểm tổng kết, quy đổi điểm hệ 4 và tính GPA theo trọng số tín chỉ. Tuy nhiên, chương trình chưa xử lý đầy đủ các trường hợp phức tạp trong thực tế như học lại, cải thiện điểm, hủy học phần, điểm chữ hoặc các quy định học vụ chi tiết khác.

Thứ năm, việc kiểm tra dữ liệu đầu vào tuy đã được thực hiện ở nhiều vị trí nhưng vẫn còn có thể cải thiện thêm. Chương trình đã kiểm tra các lỗi cơ bản như mã rỗng, trùng khóa, điểm ngoài khoảng hợp lệ hoặc dữ liệu sai định dạng trong file. Tuy nhiên, một số kiểm tra nâng cao như chuẩn hóa họ tên, kiểm tra ngày sinh theo lịch thực tế, tìm kiếm không phân biệt chữ hoa chữ thường hoặc xử lý dữ liệu tiếng Việt có dấu vẫn còn hạn chế.

Ngoài ra, chương trình hiện chủ yếu lưu dữ liệu khi người dùng chọn chức năng lưu và thoát. Do đó, nếu chương trình bị đóng đột ngột trước khi lưu, các thay đổi trong phiên làm việc hiện tại có thể chưa được ghi xuống file. Đây là hạn chế cần được xem xét nếu muốn phát triển chương trình theo hướng ổn định hơn.

Cuối cùng, mặc dù nhóm đã xây dựng một số file kiểm thử cho các module quan trọng, phạm vi kiểm thử vẫn chưa bao phủ toàn bộ mọi tình huống có thể xảy ra. Các test case hiện tại chủ yếu tập trung vào các chức năng chính, dữ liệu hợp lệ, một số dữ liệu sai định dạng và trường hợp biên cơ bản. Trong thực tế, chương trình cần được kiểm thử thêm với dữ liệu lớn hơn và nhiều tình huống nhập liệu phức tạp hơn.

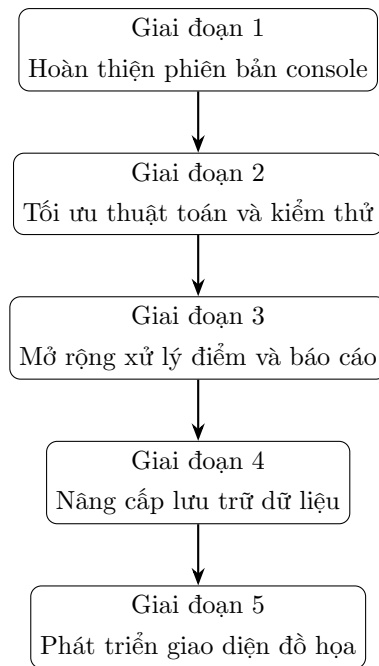
Nhìn chung, các hạn chế trên không ảnh hưởng quá lớn đến mục tiêu chính của bài tập lớn, nhưng là cơ sở để nhóm xác định hướng cải tiến trong tương lai. Nếu có thêm thời gian, chương trình có thể được mở rộng về giao diện, thuật toán, cơ chế lưu trữ dữ liệu và phạm vi kiểm thử để trở nên hoàn thiện hơn.

8.3 Hướng phát triển

Từ những kết quả đã đạt được và các hạn chế còn tồn tại, chương trình *Quản lý sinh viên* có thể được tiếp tục phát triển theo nhiều hướng khác nhau để trở nên hoàn thiện hơn. Các hướng phát triển này tập trung vào việc cải thiện giao diện, tối ưu thuật toán, nâng cao khả năng lưu trữ dữ liệu, mở rộng chức năng và tăng độ ổn định của chương trình.

Trong tương lai, chương trình có thể được phát triển theo một số hướng. Về thuật toán, nhóm có thể thay Bubble Sort bằng các thuật toán hiệu quả hơn như Quick Sort hoặc Merge Sort, đồng thời nghiên cứu áp dụng tìm kiếm nhị phân trong những trường hợp dữ liệu đã được sắp xếp phù hợp. Về lưu trữ, chương trình có thể chuyển từ file text sang cơ sở dữ liệu để tăng khả năng quản lý và truy vấn dữ liệu.

Về giao diện, chương trình có thể được mở rộng từ console sang giao diện đồ họa hoặc giao diện web để thân thiện hơn với người dùng. Ngoài ra, có thể bổ sung các chức năng thống kê kết quả học tập, xuất bảng điểm, lọc sinh viên theo học lực và phân quyền người dùng. Những hướng phát triển này giúp chương trình gần hơn với một hệ thống quản lý sinh viên thực tế.



Hình 7.3. Lộ trình phát triển chương trình trong tương lai

Trước hết, chương trình có thể được phát triển từ giao diện console sang giao diện đồ họa. Giao diện đồ họa sẽ giúp người dùng thao tác trực quan hơn thông qua các nút bấm, bảng dữ liệu, ô nhập liệu và thông báo lỗi rõ ràng. Điều này đặc biệt hữu ích khi chương trình có nhiều dữ liệu sinh viên, môn học, lớp học phần và điểm số cần hiển thị cùng lúc.

Thứ hai, cơ chế lưu trữ dữ liệu có thể được cải tiến. Hiện tại, chương trình sử dụng file text để lưu dữ liệu, phù hợp với phạm vi bài tập lớn. Trong tương lai, có thể thay thế hoặc bổ sung cơ sở dữ liệu như SQLite, MySQL hoặc PostgreSQL để quản lý dữ liệu tốt hơn. Việc sử dụng cơ sở dữ liệu sẽ giúp chương trình hỗ trợ truy vấn nhanh, đảm bảo toàn vẹn dữ liệu tốt hơn và thuận tiện hơn khi dữ liệu có quy mô lớn.

Thứ ba, các thuật toán tìm kiếm và sắp xếp có thể được tối ưu. Hiện tại, chương trình sử dụng tìm kiếm tuyến tính và Bubble Sort. Trong các phiên bản tiếp theo, có thể bổ sung tìm kiếm nhị phân đối với dữ liệu đã được sắp xếp, hoặc sử dụng các thuật toán sắp xếp hiệu quả hơn như Quick Sort, Merge Sort. Điều này giúp cải thiện tốc độ xử lý khi số lượng sinh viên hoặc bản ghi điểm tăng lên.

Thứ tư, chức năng xử lý điểm và GPA có thể được mở rộng để sát với thực tế hơn. Chương trình có thể bổ sung cách xử lý điểm chữ, học lại, học cải thiện, hủy học phần hoặc phân loại học lực chi tiết hơn. Ngoài GPA hệ 4, chương trình cũng có thể bổ sung tính điểm trung bình hệ 10 theo học kỳ, theo năm học hoặc theo toàn khóa.

Thứ năm, chương trình có thể mở rộng thêm các chức năng báo cáo. Ví dụ, hệ thống có thể thống kê danh sách sinh viên theo lớp, xếp hạng sinh viên theo GPA, lọc sinh viên theo học lực, xuất bảng điểm ra file hoặc tạo báo cáo tổng hợp theo từng lớp học phần. Những chức năng này sẽ giúp chương trình không chỉ quản lý dữ liệu mà còn hỗ trợ phân tích kết quả học tập.

Thứ sáu, việc kiểm tra dữ liệu đầu vào có thể tiếp tục được hoàn thiện. Chương trình có thể bổ sung kiểm tra ngày sinh theo lịch thực tế, chuẩn hóa họ tên, tìm kiếm không phân biệt chữ hoa chữ thường, xử lý tốt hơn dữ liệu tiếng Việt có dấu và cảnh báo chi tiết hơn khi người dùng nhập sai. Điều này giúp chương trình thân thiện hơn và hạn chế lỗi dữ liệu trong quá trình sử dụng.

Ngoài ra, nhóm có thể mở rộng phạm vi kiểm thử chương trình. Bên cạnh các file kiểm thử hiện có, chương trình nên được kiểm thử thêm với dữ liệu lớn, dữ liệu sai phức tạp và nhiều tình huống thao tác liên tiếp. Việc kiểm thử đầy đủ hơn sẽ giúp phát hiện lỗi tiềm ẩn và tăng độ tin cậy của chương trình.

Nhìn chung, các hướng phát triển trên giúp chương trình có thể tiến gần hơn đến một hệ thống quản lý sinh viên hoàn chỉnh. Mặc dù phiên bản hiện tại đã đáp ứng được yêu cầu chính của bài tập lớn, việc tiếp tục cải tiến về giao diện, lưu trữ, thuật toán, kiểm thử và chức năng nghiệp vụ sẽ giúp chương trình có tính ứng dụng cao hơn trong thực tế.

Tài liệu tham khảo

- [1] V. T. Nam, Bài giảng Kỹ thuật lập trình - Chương 1: Tổng quan [Slide PowerPoint], Hà Nội: Đại Học Bách Khoa Hà Nội, 2022.
- [2] V. T. Nam, Bài giảng Kỹ thuật lập trình - Chương 2: Cấu trúc dữ liệu và giải thuật [Slide PowerPoint], Hà Nội: Đại Học Bách Khoa Hà Nội, 2022.
- [3] V. T. Nam, Bài giảng Kỹ thuật lập trình - Chương 3: Kỹ thuật thiết kế chương trình [Slide PowerPoint], Hà Nội: Đại Học Bách Khoa Hà Nội, 2022.
- [4] V. T. Nam, Bài giảng Kỹ thuật lập trình - Chương 4: Defensive Programming Lập trình phòng ngừa [Slide PowerPoint], Hà Nội: Đại Học Bách Khoa Hà Nội, 2022.
- [5] V. T. Nam, Bài giảng Kỹ thuật lập trình - Chương 5: Gỡ lỗi và kiểm thử [Slide PowerPoint], Hà Nội: Đại Học Bách Khoa Hà Nội, 2022.
- [6] V. T. Nam, Bài giảng Kỹ thuật lập trình - Chương 6: Tối ưu mã [Slide PowerPoint], Hà Nội: Đại Học Bách Khoa Hà Nội, 2022.
- [7] V. T. Nam, Bài giảng Kỹ thuật lập trình - Chương 7: Lập trình bất đồng bộ Asynchronous programming [Slide PowerPoint], Hà Nội: Đại Học Bách Khoa Hà Nội, 2022.

PHỤ LỤC

Phụ lục A. Code hàm main

Phụ lục này trình bày code hàm `main` của chương trình. Đây là hàm điều khiển luồng hoạt động chính, bao gồm khởi tạo dữ liệu, đọc dữ liệu từ file, gọi giao diện menu, lưu dữ liệu và giải phóng bộ nhớ khi kết thúc chương trình.

```
1 #include <stdio.h>
2 #include "fileio.h"
3 #include "ui.h"
4
5 int main(void)
6 {
7     StudentArray students;
8     SubjectArray subjects;
9     CourseClassArray classes;
10    ScoreArray scores;
11
12    if (!sa_init(&students, 4)) {
13        printf("Loi khoi tao mang sinh vien.\n");
14        return 1;
15    }
16
17    if (!suba_init(&subjects, 4)) {
18        printf("Loi khoi tao mang mon hoc.\n");
19        sa_clear(&students);
20        return 1;
21    }
22
23    if (!cca_init(&classes, 4)) {
24        printf("Loi khoi tao mang lop hoc phan.\n");
25        sa_clear(&students);
26        suba_clear(&subjects);
27        return 1;
```

```
28     }
29
30     if (!sca_init(&scores, 4)) {
31         printf("Loi khoi tao mang diem.\n");
32         sa_clear(&students);
33         suba_clear(&subjects);
34         cca_clear(&classes);
35         return 1;
36     }
37
38     loadAllData(
39         &students,
40         &subjects,
41         &classes,
42         &scores
43     );
44
45     showMainMenu(
46         &students,
47         &subjects,
48         &classes,
49         &scores
50     );
51
52     saveAllData(
53         &students,
54         &subjects,
55         &classes,
56         &scores
57     );
58
59     sa_clear(&students);
60     suba_clear(&subjects);
61     cca_clear(&classes);
62     sca_clear(&scores);
63
64     return 0;
65 }
```

Listing 8.1: Code hàm main của chương trình

Phụ lục B. Một số hàm xử lý chính

B.1. Hàm thao tác với mảng động

Phần này trình bày một số hàm tiêu biểu dùng để thao tác với mảng động `StudentArray`.

B.1.1. Hàm `sa_init`

Hàm `sa_init` dùng để khởi tạo mảng động sinh viên. Hàm cấp phát vùng nhớ ban đầu cho mảng, đặt `size = 0` và thiết lập `capacity` bằng sức chứa ban đầu.

```
1 int sa_init(StudentArray* arr, int init_cap) {  
2     if (init_cap <= 0) init_cap = 4;  
3  
4     arr->data = (Student*)malloc(init_cap * sizeof(Student));  
5     if (arr->data == NULL) return 0;  
6  
7     arr->size = 0;  
8     arr->capacity = init_cap;  
9  
10    return 1;  
11 }
```

Listing 8.2: Hàm khởi tạo mảng động `StudentArray`

B.1.2. Hàm `sa_add`

Hàm `sa_add` dùng để thêm một sinh viên vào cuối mảng. Nếu mảng đã đầy, hàm gọi `sa_resize` để mở rộng vùng nhớ trước khi thêm phần tử mới.

```
1 int sa_add(StudentArray* arr, Student s) {  
2     if (arr->size >= arr->capacity) {  
3         if (!sa_resize(arr)) return 0;  
4     }  
5  
6     arr->data[arr->size] = s;  
7     arr->size++;  
8  
9     return 1;  
10 }
```

Listing 8.3: Hàm thêm sinh viên vào mảng động

B.1.3. Hàm sa_find

Hàm `sa_find` dùng để tìm sinh viên theo MSSV. Hàm duyệt lần lượt các phần tử trong mảng và so sánh MSSV bằng `strcmp`. Nếu tìm thấy, hàm trả về vị trí của sinh viên; nếu không, trả về -1.

```
1 int sa_find(StudentArray* arr, const char* mssv) {  
2     for (int i = 0; i < arr->size; i++) {  
3         if (strcmp(arr->data[i].mssv, mssv) == 0) {  
4             return i;  
5         }  
6     }  
7  
8     return -1;  
9 }
```

Listing 8.4: Hàm tìm sinh viên theo MSSV

B.1.4. Hàm sa_remove

Hàm `sa_remove` dùng để xóa sinh viên tại vị trí `index`. Sau khi xóa, các phần tử phía sau được dịch sang trái để mảng không bị rỗng ở giữa.

```
1 int sa_remove(StudentArray* arr, int index) {  
2     if (index < 0 || index >= arr->size) return 0;  
3  
4     for (int i = index; i < arr->size - 1; i++) {  
5         arr->data[i] = arr->data[i + 1];  
6     }  
7  
8     arr->size--;  
9  
10    return 1;  
11 }
```

Listing 8.5: Hàm xóa sinh viên khỏi mảng động

B.2. Hàm đọc và ghi dữ liệu

Phần này trình bày hai hàm tổng hợp dùng để đọc và ghi toàn bộ dữ liệu của chương trình. Hai hàm này được cài đặt trong file `fileio.c`.

B.2.1. Hàm `loadAllData`

Hàm `loadAllData` dùng để đọc toàn bộ dữ liệu từ các file text vào bộ nhớ. Hàm lần lượt gọi các hàm đọc dữ liệu sinh viên, môn học, lớp học phần và điểm số.

```
1 void loadAllData(StudentArray* students ,
2                   SubjectArray* subjects ,
3                   CourseClassArray* classes ,
4                   ScoreArray* scores) {
5     loadStudents(students , STUDENT_FILE);
6     loadSubjects(subjects , SUBJECT_FILE);
7     loadCourseClasses(classes , COURSE_CLASS_FILE);
8     loadScores(scores , SCORE_FILE);
9 }
```

Listing 8.6: Hàm đọc toàn bộ dữ liệu từ file

Trong hàm trên, mỗi loại dữ liệu được đọc từ một file riêng. Dữ liệu sinh viên được nạp vào `StudentArray`, dữ liệu môn học được nạp vào `SubjectArray`, dữ liệu lớp học phần được nạp vào `CourseClassArray` và dữ liệu điểm số được nạp vào `ScoreArray`.

B.2.2. Hàm `saveAllData`

Hàm `saveAllData` dùng để ghi toàn bộ dữ liệu hiện có trong bộ nhớ xuống các file text tương ứng. Hàm này được gọi khi chương trình kết thúc để lưu lại dữ liệu sau quá trình sử dụng.

```
1 void saveAllData(StudentArray* students ,
2                   SubjectArray* subjects ,
3                   CourseClassArray* classes ,
4                   ScoreArray* scores) {
5     saveStudents(students , STUDENT_FILE);
6     saveSubjects(subjects , SUBJECT_FILE);
7     saveCourseClasses(classes , COURSE_CLASS_FILE);
8     saveScores(scores , SCORE_FILE);
9 }
```

Listing 8.7: Hàm ghi toàn bộ dữ liệu xuống file

Hàm `saveAllData` giúp quá trình lưu dữ liệu được thực hiện tập trung. Thay vì gọi từng hàm ghi dữ liệu ở nhiều vị trí khác nhau, chương trình chỉ cần gọi một hàm duy nhất để lưu toàn bộ danh sách sinh viên, môn học, lớp học phần và điểm số.

B.3. Hàm xử lý điểm và GPA

Phần này trình bày một số hàm tiêu biểu dùng để tính điểm tổng kết, quy đổi điểm hệ 4 và tính GPA của sinh viên.

B.3.1. Hàm `calculateDiemTK`

Hàm `calculateDiemTK` dùng để tính điểm tổng kết từ điểm quá trình và điểm cuối kỳ.

```
1 float calculateDiemTK(float diemQT, float diemCK)
2 {
3     return (diemQT + diemCK) / 2.0f;
4 }
```

Listing 8.8: Hàm tính điểm tổng kết

B.3.2. Hàm `convertToHe4`

Hàm `convertToHe4` dùng để quy đổi điểm tổng kết hệ 10 sang điểm hệ 4 theo các mốc điểm đã quy định.

```
1 float convertToHe4(float diemTK)
2 {
3     if (diemTK >= 8.5f)
4         return 4.0f;
5
6     if (diemTK >= 8.0f)
7         return 3.5f;
8
9     if (diemTK >= 7.0f)
10        return 3.0f;
11
12    if (diemTK >= 6.5f)
13        return 2.5f;
14
15    if (diemTK >= 5.5f)
16        return 2.0f;
17
18    if (diemTK >= 5.0f)
```

```
19         return 1.5f;
20
21     if (diemTK >= 4.0f)
22         return 1.0f;
23
24     return 0.0f;
25 }
```

Listing 8.9: Hàm quy đổi điểm hệ 10 sang hệ 4

B.3.3. Hàm calculateStudentGPA4

Hàm `calculateStudentGPA4` dùng để tính GPA hệ 4 của một sinh viên. Hàm duyệt qua các bản ghi điểm của sinh viên, tìm lớp học phần và môn học tương ứng để lấy số tín chỉ, sau đó tính GPA theo trọng số tín chỉ.

```
1 float calculateStudentGPA4(
2     const char* mssv,
3     ScoreArray* scores,
4     CourseClassArray* classes,
5     SubjectArray* subjects
6 )
7 {
8     float tongDiem = 0.0f;
9     int tongTinChi = 0;
10
11     for(int i = 0; i < scores->size; i++)
12     {
13         ScoreRecord sc = scores->data[i];
14
15         if(strcmp(sc.mssv, mssv) != 0)
16             continue;
17
18         int classIndex =
19             cca_find(
20                 classes,
21                 sc.maLHP
22             );
23
24         if(classIndex == -1)
25             continue;
26
27         CourseClass cc =
```

```
28         classes->data[classIndex];
29
30     int subjectIndex =
31         suba_find(
32             subjects,
33             cc.maHP
34         );
35
36     if(subjectIndex == -1)
37         continue;
38
39     Subject sub =
40         subjects->data[subjectIndex];
41
42     tongDiem +=
43         sc.diemHe4 *
44         sub.soTinChi;
45
46     tongTinChi +=
47         sub.soTinChi;
48 }
49
50 if(tongTinChi == 0)
51     return 0.0f;
52
53 return tongDiem / tongTinChi;
54 }
```

Listing 8.10: Hàm tính GPA hệ 4 của sinh viên

B.3.4. Hàm calculateStudentGPA10

Hàm calculateStudentGPA10 có cách xử lý tương tự hàm tính GPA hệ 4, nhưng sử dụng điểm tổng kết diemTK để tính GPA theo hệ 10.

```
1 float calculateStudentGPA10(
2     const char* mssv,
3     ScoreArray* scores,
4     CourseClassArray* classes,
5     SubjectArray* subjects
6 )
7 {
8     float tongDiem = 0.0f;
```

```
9      int tongTinChi = 0;
10
11      for(int i = 0; i < scores->size; i++)
12      {
13          ScoreRecord sc = scores->data[i];
14
15          if(strcmp(sc.mssv, mssv) != 0)
16              continue;
17
18          int classIndex =
19              cca_find(
20                  classes,
21                  sc.maLHP
22              );
23
24          if(classIndex == -1)
25              continue;
26
27          CourseClass cc =
28              classes->data[classIndex];
29
30          int subjectIndex =
31              suba_find(
32                  subjects,
33                  cc.maHP
34              );
35
36          if(subjectIndex == -1)
37              continue;
38
39          Subject sub =
40              subjects->data[subjectIndex];
41
42          tongDiem +=
43              sc.diemTK *
44              sub.soTinChi;
45
46          tongTinChi +=
47              sub.soTinChi;
48      }
49
```

```
50     if(tongTinChi == 0)
51         return 0.0f;
52
53     return tongDiem / tongTinChi;
54 }
```

Listing 8.11: Hàm tính GPA hệ 10 của sinh viên

B.4. Hàm tìm kiếm và sắp xếp

Phần này trình bày một số hàm tiêu biểu dùng để tìm kiếm và sắp xếp sinh viên.

B.4.1. Hàm `searchStudentByMSSV`

Hàm `searchStudentByMSSV` dùng để tìm sinh viên theo MSSV. Nếu tìm thấy, hàm trả về con trỏ đến sinh viên; nếu không tìm thấy, hàm trả về `NULL`.

```
1 Student* searchStudentByMSSV(
2     StudentArray* students,
3     const char* mssv
4 )
5 {
6     int idx =
7         sa_find(students, mssv);
8
9     if(idx == -1)
10         return NULL;
11
12     return &students->data[idx];
13 }
```

Listing 8.12: Hàm tìm sinh viên theo MSSV

B.4.2. Hàm `sortStudentByMSSV`

Hàm `sortStudentByMSSV` dùng để sắp xếp sinh viên tăng dần theo MSSV bằng thuật toán Bubble Sort.

```
1 void sortStudentByMSSV(StudentArray* students)
2 {
3     for(int i = 0; i < students->size - 1; i++)
4     {
5         for(int j = 0; j < students->size - i - 1; j++)
6         {
```

```

7         if(strcmp(
8             students->data[j].mssv,
9             students->data[j + 1].mssv
10        ) > 0)
11        {
12            Student temp =
13                students->data[j];
14
15            students->data[j] =
16                students->data[j + 1];
17
18            students->data[j + 1] =
19                temp;
20        }
21    }
22 }
23 }

```

Listing 8.13: Hàm sắp xếp sinh viên theo MSSV

B.4.3. Hàm sortStudentByName

Hàm `sortStudentByName` dùng để sắp xếp sinh viên tăng dần theo họ tên. Hàm so sánh trường `hoTen` của hai sinh viên liền kề và hoán đổi nếu sai thứ tự.

```

1 void sortStudentByName(StudentArray* students)
2 {
3     for(int i = 0; i < students->size - 1; i++)
4     {
5         for(int j = 0; j < students->size - i - 1; j++)
6         {
7             if(strcmp(
8                 students->data[j].hoTen,
9                 students->data[j + 1].hoTen
10            ) > 0)
11            {
12                Student temp =
13                    students->data[j];
14
15                students->data[j] =
16                    students->data[j + 1];
17
18                students->data[j + 1] =

```

```
19         temp;
20     }
21 }
22 }
23 }
```

Listing 8.14: Hàm sắp xếp sinh viên theo họ tên

B.4.4. Hàm sortStudentByGPA

Hàm `sortStudentByGPA` được cài đặt tương tự Bubble Sort, nhưng so sánh GPA hệ 10 đã tính từ dữ liệu điểm và sắp xếp theo thứ tự giảm dần.

```
1 void sortStudentByGPA(
2     StudentArray* students,
3     ScoreArray* scores,
4     CourseClassArray* classes,
5     SubjectArray* subjects
6 )
7 {
8     for(int i = 0; i < students->size - 1; i++)
9     {
10         for(int j = 0; j < students->size - i - 1; j++)
11         {
12             float gpa1 =
13                 calculateStudentGPA10(
14                     students->data[j].mssv,
15                     scores,
16                     classes,
17                     subjects
18                 );
19
20             float gpa2 =
21                 calculateStudentGPA10(
22                     students->data[j + 1].mssv,
23                     scores,
24                     classes,
25                     subjects
26                 );
27
28             if(gpa1 < gpa2)
29             {
30                 Student temp =
```

```
31         students->data[j];
32
33         students->data[j] =
34             students->data[j + 1];
35
36         students->data[j + 1] =
37             temp;
38     }
39 }
40 }
41 }
```

Listing 8.15: Hàm sắp xếp sinh viên theo GPA

Phụ lục C. File dữ liệu mẫu

Phụ lục này trình bày một số file dữ liệu mẫu được sử dụng trong chương trình. Các file dữ liệu được lưu dưới dạng text, mỗi dòng là một bản ghi và các trường được phân tách bằng ký tự |.

C.1. File students.txt

```
MSSV|HoTen|Lop|Birthday
202400000|Nguyen Van Toan|K69-MI1-01|15/08/2006
202400001|Tran Quan Anh|K69-MI1-02|20/03/2006
202400002|Le Hoang Thai|K69-MI1-03|05/11/2005
202400003|Vu Thi Hoa|K69-MI1-04|22/09/2002
202400004|Nguyen Thi Mai|K69-MI1-01|10/05/2004
202400010|Tran Van Nam|K69-MI1-02|18/02/2004
```

C.2. File subjects.txt

```
MaHP|TenHP|SoTinChi
MI3310|Ky Thuat Lap Trinh|2
MI3060|Cau Truc Du Lieu & Thuat Toan|3
MI3090|Co So Du Lieu|3
MI2020|Kien truc may tinh|2
MI3040|Toan Roi Rac|3
```


C.3. File course_classes.txt

MaLHP|MaHP|HocKy|NamHoc

169313|MI3310|1|2025

169307|MI3060|2|2025

169320|MI3090|3|2025

169400|MI2020|1|2025

169401|MI3040|2|2025

C.4. File scores.txt

MSSV|MaLHP|DiemQT|DiemCK|DiemTK|DiemHe4

202400000|169313|8.50|7.00|7.75|3.00

202400000|169307|8.00|9.00|8.50|4.00

202400000|169320|6.50|7.00|6.75|2.50

202400000|169400|9.00|8.00|8.50|4.00

202400000|169401|7.50|8.00|7.75|3.00

202400001|169307|9.00|8.50|8.75|4.00

202400001|169313|7.00|8.00|7.50|3.00

202400001|169320|8.00|7.00|7.50|3.00

202400001|169400|6.00|6.50|6.25|2.00

202400002|169320|7.00|7.50|7.25|3.00

202400002|169313|9.00|9.50|9.25|4.00

202400002|169307|5.50|6.00|5.75|2.00

202400002|169401|8.50|9.00|8.75|4.00

202400003|169313|6.00|5.50|5.75|2.00

202400003|169307|7.00|6.50|6.75|2.50

202400003|169320|8.00|8.50|8.25|3.50

202400004|169313|5.00|4.50|4.75|1.00

202400004|169400|9.50|10.00|9.75|4.00

202400004|169401|7.00|7.50|7.25|3.00

202400010|169307|4.00|3.50|3.75|0.00

202400010|169313|6.50|7.00|6.75|2.50

202400010|169320|8.00|7.00|7.50|3.00

Phụ lục D. Một số kết quả kiểm thử trên giao diện console

D.1. Phân loại kiểm thử

Bảng 8.1: Phân loại kiểm thử áp dụng trong chương trình

Loại kiểm thử	Áp dụng trong đề tài
Internal Testing	Nhóm thiết kế các bộ dữ liệu để kiểm tra chương trình, gồm dữ liệu hợp lệ, dữ liệu trùng khóa, điểm ngoài khoảng, file thiếu trường và dữ liệu sai quan hệ khóa ngoại.
External Testing	Nhóm xây dựng các file kiểm thử như <code>test_types.c</code> , <code>test_arrays.c</code> , <code>test_fileio_unit.c</code> , <code>test_gpa.c</code> và <code>test_fileio.c</code> để chương trình có thể tự kiểm tra một số module quan trọng.
Black-box Testing	Các test case thao tác qua giao diện console được thực hiện dựa trên yêu cầu chức năng. Người kiểm thử nhập dữ liệu, quan sát kết quả đầu ra và so sánh với kết quả mong đợi mà không cần xét chi tiết mã nguồn.
White-box Testing	Các file kiểm thử tự động gọi trực tiếp đến các hàm bên trong chương trình như hàm thao tác mảng động, hàm đọc/ghi file, hàm tính điểm và hàm tính GPA. Cách kiểm thử này dựa trên hiểu biết về cấu trúc và logic bên trong mã nguồn.
Test tĩnh	Nhóm kiểm tra tài liệu, mã nguồn, cấu trúc thư mục, khai báo hàm, định dạng file dữ liệu và các ràng buộc dữ liệu mà không cần thực thi chương trình.
Test động	Nhóm chạy trực tiếp chương trình <code>qlsv.exe</code> để kiểm thử giao diện, đồng thời chạy các file kiểm thử tự động để kiểm tra kết quả khi chương trình thực thi.

D.2. Một số hình ảnh kiểm thử

```
===== QUAN LY SINH VIEN =====
1. Hien thi danh sach sinh vien
2. Them sinh vien
3. Sua sinh vien
4. Xoa sinh vien
5. Tim sinh vien
6. Sap xep danh sach sinh vien
0. Quay lai
Nhap lua chon: 2
Nhap MSSV: 202499999
Nhap ho ten: Nguyen Test UI
Nhap lop: MI1-01-K69
Nhap ngay sinh DD/MM/YYYY: 22/082006
Ngay sinh khong hop le. Vi du dung: 15/08/2006
Nhap ngay sinh DD/MM/YYYY: 22/08/2006
Them sinh vien thanh cong.
```

Hình 8.1: TC01 — Thêm sinh viên hợp lệ

```
===== QUAN LY SINH VIEN =====
1. Hien thi danh sach sinh vien
2. Them sinh vien
3. Sua sinh vien
4. Xoa sinh vien
5. Tim sinh vien
6. Sap xep danh sach sinh vien
0. Quay lai
Nhap lua chon: 2
Nhap MSSV: 202499999
MSSV da ton tai. Vui long nhap MSSV khac.
Nhap MSSV: |
```

Hình 8.2: TC02 — Thêm sinh viên trùng MSSV

```
===== QUAN LY MON HOC =====
1. Hien thi danh sach mon hoc
2. Them mon hoc
3. Sua mon hoc
4. Xoa mon hoc
5. Tim mon hoc
0. Quay lai
Nhap lua chon: 2
Nhap MaHP: TEST01
Nhap ten hoc phan: Mon Test UI
Nhap so tin chi: 3
Them mon hoc thanh cong.
```

Hình 8.3: TC03 — Thêm môn học hợp lệ

```

===== QUAN LY MON HOC =====
1. Hien thi danh sach mon hoc
2. Them mon hoc
3. Sua mon hoc
4. Xoa mon hoc
5. Tim mon hoc
0. Quay lai
Nhap lua chon: 2
Nhap MaHP: TEST01
MaHP da ton tai.
Nhap MaHP: |

```

Hình 8.4: TC04 — Thêm môn học trùng mã

```

===== QUAN LY LOP HOC PHAN =====
1. Hien thi danh sach lop hoc phan
2. Them lop hoc phan
3. Sua lop hoc phan
4. Xoa lop hoc phan
5. Tim lop hoc phan
0. Quay lai
Nhap lua chon: 2
Nhap MaLHP: TESTLHP1
Nhap MaHP: TEST01
Nhap hoc ky (1-3): 1
Nhap nam hoc: 2026
Them lop hoc phan thanh cong.

```

Hình 8.5: TC05 — Thêm lớp học phần hợp lệ

```

===== QUAN LY LOP HOC PHAN =====
1. Hien thi danh sach lop hoc phan
2. Them lop hoc phan
3. Sua lop hoc phan
4. Xoa lop hoc phan
5. Tim lop hoc phan
0. Quay lai
Nhap lua chon: 2
Nhap MaLHP: TESTLHP1
MaLHP da ton tai.
Nhap MaLHP: |

```

Hình 8.6: TC06 — Thêm lớp học phần với MaHP không tồn tại

```

===== QUAN LY DIEM =====
1. Hien thi danh sach diem
2. Nhap diem
3. Cap nhat diem
4. Tim diem
0. Quay lai
Nhap lua chon: 2
Nhap MSSV: 202499999
Nhap MaLHP: TESTLHP01
Nhap diem qua trinh (0-10): 10
Nhap diem cuoi ky (0-10): 10
Them diem thanh cong. DiemTK = 10.00, He4 = 4.00

```

Hình 8.7: TC07 — Nhập điểm hợp lệ

```
===== QUAN LY DIEM =====
1. Hien thi danh sach diem
2. Nhap diem
3. Cap nhat diem
4. Tim diem
0. Quay lai
Nhap lua chon: 2
Nhap MSSV: 2024199998
Nhap MaLHP: TESTLHP01
Nhap diem qua trinh (0-10): 11
Gia tri phai nam trong khoang [0.00, 10.00].
Nhap diem qua trinh (0-10): -1
Gia tri phai nam trong khoang [0.00, 10.00].
Nhap diem qua trinh (0-10): |
```

Hình 8.8: TC08 — Nhập điểm ngoài khoảng

```
===== QUAN LY DIEM =====
1. Hien thi danh sach diem
2. Nhap diem
3. Cap nhat diem
4. Tim diem
0. Quay lai
Nhap lua chon: 2
Nhap MSSV: 202488888
MSSV khong ton tai trong danh sach sinh vien.
Nhap MSSV: |
```

Hình 8.9: TC09 — Nhập điểm cho sinh viên không tồn tại

```
===== QUAN LY DIEM =====
1. Hien thi danh sach diem
2. Nhap diem
3. Cap nhat diem
4. Tim diem
0. Quay lai
Nhap lua chon: 2
Nhap MSSV: 202499999
Nhap MaLHP: TESTLHP01
Ban ghi diem da ton tai. Hay dung chuc nang cap nhat diem.
```

Hình 8.10: TC10 — Nhập điểm trùng

```
===== QUAN LY DIEM =====
1. Hien thi danh sach diem
2. Nhap diem
3. Cap nhat diem
4. Tim diem
0. Quay lai
Nhap lua chon: 3
Nhap MSSV: 202499999
Nhap MaLHP: TESTLHP01
Nhap diem qua trinh moi (0-10): 9
Nhap diem cuoi ky moi (0-10): 7
Cap nhat diem thanh cong. DiemTK = 8.00, He4 = 3.50
```

Hình 8.11: TC11 — Cập nhật điểm

```

===== BAO CAO =====
1. Bang diem cua mot sinh vien
2. Bang diem cua mot lop hoc phan
0. Quay lai
Nhap lua chon: 1
Nhap MSSV can xem bang diem: 202499999

Bang diem sinh vien: 202499999 - Nguyen Test UI
MaLHP      | DiemQT   | DiemCK   | DiemTK   | He4
-----
TESTLHP01  | 9.00     | 7.00     | 8.00     | 3.50

GPA he 10 : 8.00
GPA he 4  : 3.50
Hoc luc   : Gioi

```

Hình 8.12: TC12 — Xếp loại học lực

```

===== QUAN LY SINH VIEN =====
1. Hien thi danh sach sinh vien
2. Them sinh vien
3. Sua sinh vien
4. Xoa sinh vien
5. Tim sinh vien
6. Sap xep danh sach sinh vien
0. Quay lai
Nhap lua chon: 5

1. Tim theo MSSV
2. Tim theo ho ten
3. Tim theo lop
Nhap lua chon: 1
Nhap tu khoa: 202400000

MSSV      | Ho ten                | Lop      | Birthday
-----
202400000 | Nguyen Van Toan       | K69-MI1-01 | 15/08/2006

```

Hình 8.13: TC13 — Tìm sinh viên tồn tại

```

===== QUAN LY SINH VIEN =====
1. Hien thi danh sach sinh vien
2. Them sinh vien
3. Sua sinh vien
4. Xoa sinh vien
5. Tim sinh vien
6. Sap xep danh sach sinh vien
0. Quay lai
Nhap lua chon: 5

1. Tim theo MSSV
2. Tim theo ho ten
3. Tim theo lop
Nhap lua chon: 1
Nhap tu khoa: 9999999

MSSV      | Ho ten                | Lop      | Birthday
-----
Khong tim thay ket qua phu hop.

```

Hình 8.14: TC14 — Tìm sinh viên không tồn tại

```

===== BAO CAO =====
1. Bang diem cua mot sinh vien
2. Bang diem cua mot lop hoc phan
0. Quay lai
Nhap lua chon: 2
Nhap MaLHP can xem bang diem: 169313
MSSV      | HoTen                | DiemQT   | DiemCK   | DiemTK   | He4
-----
202400000 | Nguyen Van Toan      | 8.50     | 7.00     | 7.75     | 3.00
202400001 | Tran Quan Anh        | 7.00     | 8.00     | 7.50     | 3.00
202400002 | Le Hoang Thai        | 9.00     | 9.50     | 9.25     | 4.00
202400003 | Vu Thi Hoa           | 6.00     | 5.50     | 5.75     | 2.00
202400004 | Nguyen Thi Mai       | 5.00     | 4.50     | 4.75     | 1.00
202400010 | Tran Van Nam         | 6.50     | 7.00     | 6.75     | 2.50

===== BAO CAO =====
1. Bang diem cua mot sinh vien
2. Bang diem cua mot lop hoc phan
0. Quay lai
Nhap lua chon: |

```

Hình 8.15: TC15 — Hiển thị bảng điểm lớp học phần

```
===== SAP XEP SINH VIEN =====
1. Sap xep theo MSSV
2. Sap xep theo ho ten
3. Sap xep theo GPA
0. Quay lai
Nhap lua chon: 3
Da sap xep danh sach sinh vien theo GPA.

===== DANH SACH SINH VIEN =====
MSSV      | Ho ten      | Lop      | Birthday
-----|-----|-----|-----
202400000 | Nguyen Van Toan | K69-MI1-01 | 15/08/2006
202400001 | Tran Quan Anh  | K69-MI1-02 | 20/03/2006
202400002 | Le Hoang Thai  | K69-MI1-03 | 05/11/2005
202400004 | Nguyen Thi Mai  | K69-MI1-01 | 10/05/2004
202400003 | Vu Thi Hoa     | K69-MI1-04 | 22/09/2002
202400010 | Tran Van Nam   | K69-MI1-02 | 18/02/2004
Tong so sinh vien: 6
```

Hình 8.16: TC16 — Sắp xếp sinh viên theo GPA

Phụ lục E. Github Repository

https://github.com/PKT-zZ/QLSV_MI3310.git